Learning a Decision Policy from Synthetic Data: Pre-Registered QLoRA Fine-Tuning of Open-Weight Models for a Student AI Operating System

Tejas Naik
The Ohio State University
naik.tejas11@gmail.com

October 2026

Code and research records: https://github.com/TejasNaik24/Kleos-Models
Data pipeline: https://github.com/TejasNaik24/Kleos-Training-Data KLEOS: https://kleos-cs.vercel.app

Abstract

KLEOS is an AI operating system for computer science students whose language models make judgment calls: ranking competing work, resolving conflicting records and deciding when to decline. This paper reports a pre-registered study of teaching one such decision policy to small open-weight models from synthetic data on free GPUs. All 1,350 examples were generated from 18 scenario families and answered by one of 12 explicit policies, so every answer is correct by construction; the 349 examples in a held-out input format form the benchmark (271 answerable, 78 should-decline, 78 perturbation groups). Mistral-Nemo-Instruct 12B (KLEOS Hermes) and Ministral 3 14B (KLEOS Logos: an Instruct variant, then a Reasoning variant trained on policy-derived reasoning traces) were fine-tuned with QLoRA on 820 examples under nine hypotheses registered before the runs, six of them with decision rules, with every deviation logged. Fine-tuning raised Hermes from 0.4755 to 0.8051 (+0.3295, 95% CI 0.3068 to 0.3521, all seven tasks $p < 0.001$), replicating a Ministral-8B pilot (0.4744 to 0.8015). Logos v0.0.1 against Hermes on the answerable items was inconclusive (−0.0168, cluster 95% CI −0.0546 to +0.0177); Logos v0.0.2 was measurably better (+0.0409, cluster 95% CI +0.0040 to +0.0780, $p = 0.031$), while its should-decline gain (+0.1020, CI −0.0018 to +0.2062) was not significant by groups. Each model is one training run, the v0.0.2 comparison confounds three changes, no model produced the held-out JSON format, and abstention was learned by scenario family rather than from evidence. Both models serve from Hugging Face ZeroGPU Spaces and reproduce the frozen evaluation byte for byte on 9 of 9 and 8 of 9 checks; the code, records and data pipeline are public.

1 Introduction

KLEOS is an AI operating system for computer science students (https://kleos-cs.vercel.app). Beneath its planning, memory and research features sits a layer of small decisions that a language model makes many times a day: which of several competing items deserves attention first, which of two contradictory records to trust, whether a request is specific enough to act on, which items belong in a briefing, and which sit in a workspace the user has not opened. These decisions are frequent and consequential, and a general-purpose model prompted with instructions does not reliably make them the way the product requires (the orchestrated base models score 0.4469 to 0.4755 on the benchmark, Section 6).

KLEOS therefore defines its decisions as an explicit policy. Each candidate item is scored as the product of a deadline term, an evidence weight and an impact weight; items whose scores fall within a relative margin are declined rather than guessed; an explicit statement by the user outranks an inferred record unless it is stale; items outside the active workspace are flagged rather than acted on; and an ambiguous request is answered with a question. The policy is deterministic, so the correct answer to any scenario is computable. That makes two things possible: a synthetic training set in which every answer is correct by construction, and a benchmark on which a model's judgment is measured against the policy rather than against human raters.

This paper asks whether a small open-weight model can learn that policy from such data under conditions a student can afford: free Colab and Kaggle T4 GPUs, QLoRA adapters rather than full fine-tuning, and no proprietary models.

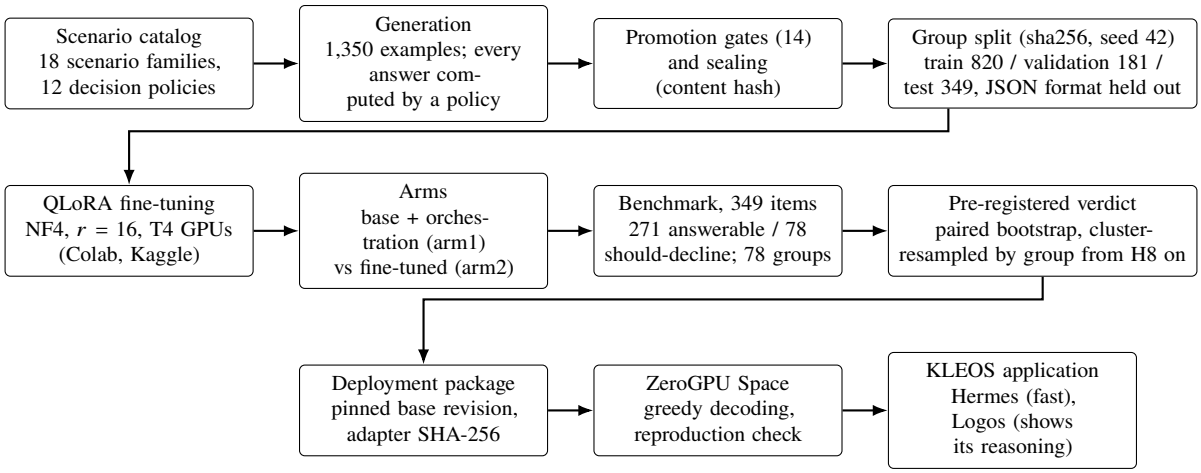


Figure 1: The pipeline. The data pipeline (top row) lives in the companion repository and seals each release with a content hash; the training, evaluation and serving code (rows two and three) live in the KLEOS Models repository. The test split is the sealed benchmark; it was never used for selection.

It reports two production models. KLEOS Hermes is a QLoRA adapter on Mistral-Nemo-Instruct 12B. KLEOS Logos is an adapter on Ministral 3 14B, in a first version on the Instruct checkpoint and a second on the Reasoning checkpoint, trained on reasoning traces derived from the same policy. Every hypothesis that was run was written down with its metric and decision rule before the run, every deviation from the protocol is dated and logged, and negative and inconclusive results are reported with the same precision as positive ones. The design follows the pre-registration literature [Nosek et al., 2018, van Miltenburg et al., 2021, Bertinetto et al., 2021] and recent guidance on interval estimates for language-model evaluations [Miller, 2024].

Three findings follow. First, fine-tuning on 820 synthetic examples moved the benchmark score of Hermes from 0.4755 to 0.8051 (difference +0.3295, 95% CI 0.3068 to 0.3521, every one of the seven tasks $p < 0.001$), replicating a pilot on Ministral-8B (0.4744 to 0.8015, seven of seven tasks). Second, moving to the stronger Ministral 3 base did not by itself produce a measurably better model: Logos v0.0.1 differed from Hermes by −0.0168 on the answerable items (cluster 95% CI −0.0546 to +0.0177), an inconclusive result under the pre-registered rule. Third, Logos v0.0.2, trained on the data release that adds policy-derived reasoning traces, was measurably better than Hermes on the answerable items (+0.0409, cluster 95% CI +0.0040 to +0.0780, $p = 0.031$); its gain on the should-decline items did not reach significance by groups. Each model is a single training run, so these intervals cover evaluation noise only, and the v0.0.2 comparison confounds three changes made together.

The contributions are:

- A sealed synthetic dataset and benchmark for a product decision policy: 1,350 examples over 18 scenario families and 12 policies, identified by content hashes and rebuilt byte for byte by the public pipeline, whose 349-item test split is byte-identical across the two releases used here (Section 3).
- A pre-registered protocol for fine-tuning experiments on free compute: dated hypotheses H1 to H9, six of them with decision rules, a cluster-bootstrap amendment adopted before the second model line was trained, and a log of thirteen deviations, D1 to D13, each with its effect on the result (Section 5, Appendices D and F).
- The H1 result and its replication across two base models, with sub-score, consistency and abstention analyses (Sections 6 and 7).
- The H8 and H9 comparisons between model lines, with their confounds declared before the runs (Sections 6 and 7).
- A serving path on free ZeroGPU Spaces with pinned base revisions and adapter hashes, verified by byte-for-byte reproduction of the frozen evaluation outputs: 9 of 9 for Hermes and 8 of 9 for Logos (Section 8).

Figure 1 shows how the data pipeline, the training and evaluation harness and the serving path fit together, and Section 2 places the work among its neighbours. Throughout, every number is copied from a frozen record in the public repository, and each table names the record it reproduces. Analyses are labelled pre-registered, with their hypothesis identifier, or exploratory. Section 9 collects the limitations and Section 10 states what can be reproduced, and by whom.

2 Related work

Parameter-efficient and quantized fine-tuning. LoRA [Hu et al., 2022] freezes a pretrained model and trains low-rank updates to selected weight matrices. QLoRA [Dettmers et al., 2023] stores the frozen weights in a 4-bit NormalFloat format with double quantization and keeps optimizer state in paged memory, which is what lets a 12B or 14B model train on a single T4 (nominally 16 GB). The 8-bit optimizer [Dettmers et al., 2022b] and the int8 matrix-multiplication kernels [Dettmers et al., 2022a] are the bitsandbytes primitives beneath it; PEFT [Mangrulkar et al., 2022] and Transformers [Wolf et al., 2020] implement the recipe. This work uses that recipe as published, with no hyperparameter search, and contributes measurements of it on a narrow policy task rather than a new method.

Synthetic and distilled supervision. Self-Instruct [Wang et al., 2023] and Alpaca [Taori et al., 2023] generate instruction data from a language model and fine-tune a smaller one on it. Chain-of-thought prompting [Wei et al., 2022] showed that intermediate reasoning text improves answers, and later work trains small models on rationales produced by a larger teacher [Ho et al., 2023, Hsieh et al., 2023]; DeepSeek-R1 [Guo et al., 2025] trains reasoning with reinforcement learning and distils the resulting traces into smaller open models. The data here differs in one respect: both the answers and the reasoning traces are computed by a deterministic policy, not sampled from a teacher model. The supervision therefore carries no model noise and no license entanglement, at the cost that the benchmark measures how well a model learns those policies, not general reasoning.

Statistics for model comparison. Dror et al. [2018] survey significance testing in natural language processing, and Efron and Tibshirani [1993] is the standard reference for the bootstrap used here. When evaluation items come in groups, resampling must respect the grouping [Field and Welsh, 2007], and inference with few clusters over-rejects [Cameron et al., 2008]. Miller [2024] makes the same points for language-model evaluations and recommends clustered standard errors for grouped questions and paired differences when comparing models. Dodge et al. [2020] measured how much fine-tuning results vary with the random seed, and Agarwal et al. [2021] argue for interval estimates over point estimates when runs are few. This study adopts paired resampling and, from its second model line on, cluster resampling by benchmark group; its single-run design is its main statistical limitation (Section 9).

Pre-registration and reporting. Nosek et al. [2018] set out the case for registering hypotheses and analysis plans before data are seen; van Miltenburg et al. [2021] adapt it to natural language processing and Bertinetto et al. [2021] to machine learning more broadly. Model cards [Mitchell et al., 2019] ask that a released model carry its evaluation conditions and limitations, and Lipton and Steinhardt [2019] ask that explanation be kept apart from speculation. Altman and Bland [1995] is the reminder that a null result is an estimate with an interval, not evidence of equivalence. The protocol used here is a lightweight version of those proposals: hypotheses, metrics and decision rules committed to a public repository before each run, with the commit identifier as the timestamp, and every outcome reported, including the inconclusive one.

3 The decision task and the data

3.1 The tasks and the example format

KLEOS routes its judgment calls through seven decision tasks: notification prioritization ("rank competing items by urgency, evidence and impact"), tool routing ("choose the right tool, or none"), mission-control briefing ("synthesize a structured briefing from context"), context prioritization ("select and order the most relevant context"), recommendation generation ("recommend actions grounded in supplied evidence"), memory-conflict resolution ("resolve contradictions using recency and reliability") and workspace reasoning ("reason within workspace scoping rules"). An example is a conversation: a system message at index 0 that states the model's role and the rule it must apply; a user message that presents the items in one of four formats, bullets, prose, JSON or a Slack-style thread; and the reference assistant answer. The answer either commits to a ranking, with a one-line rationale per item, or declines and says what it needs; the reference carries the label of the factor that decided it, and in release v0.0.7 every training and validation answer also names that factor in a closing line. The 1,350 examples in the releases used here span four domains (career 335, coursework 341, projects 340, research 334) and the four formats (bullets 473, prose 464, JSON 349, Slack-style 64). Appendix B shows the format on a public development fixture.

3.2 The policy

Each item is scored as

$$\text{score} = \text{deadline_score} \times \text{evidence_weight} \times \text{impact_weight}, \qquad \text{deadline_score} = \frac{1}{1 + \text{days to the deadline}}, \quad (1)$$

with evidence weights `confirmed` 1.0, `corroborated` 0.8, `reported` 0.55, `single_source` 0.35, `unverified` 0.2 and `contradicted` 0.1, and impact weights `high` 1.0, `medium` 0.6 and `low` 0.3. Ties are broken by the item key, so every ranking is total and reproducible. Twelve named policies combine the score with checks; Table 14 in Appendix C lists each with what it orders by, when it declines and the deciding factor it names. Three named constants govern the checks: a close call is a relative separation $|a - b|/\max(a, b)$ below 0.15 between the top two scores; an explicit statement is stale after 30 days; and a record that challenges an explicit statement must carry evidence of at least 0.8 (`corroborated`). "Adequate" evidence means a weight of at least 0.55 (`reported`), and an "explicit statement" is an item graded `confirmed`. An example's difficulty is computed from the policy's own ordering rather than copied from an axis. It depends on the first sort component on which the top two candidates differ: a gating criterion such as scope or adequacy makes the example easy; the primary ranking component makes it easy at a relative gap of at least 0.20 and medium below; a later component makes it medium or hard by the same threshold; and an order that falls to the item key makes it hard. The 0.20 threshold is the smallest relative gap between adjacent evidence grades, $(1.0 - 0.8)/1.0$. The policy table names eleven distinct deciding-factor labels, of which the benchmark's label set uses nine: six commit labels (`deadline`, `evidence`, `impact`, `scope`, `explicit_statement`, `least_privilege`) and five decline labels (`insufficient_separation`, `missing_input`, `request_ambiguous`, `stale_explicit_conflict`, `ask_before_crossing`). Appendix C lists which policy produces each label; no record glosses the labels in prose, so that table is assembled from the policy table.

3.3 Scenario families, perturbations and groups

Table 1 lists the 18 scenario families. Each binds a task to one policy, draws fictional entities from a pool, and varies urgency (deadline pools per urgency class: critical 0, 1 or 2 days; high 1, 2, 3 or 5; medium 4, 7, 10 or 14; low 14, 21, 30 or 45), context length (0, 2 or 4 distractor items) and difficulty spacing (2, 1 or 0 days between adjacent deadlines). A family produces $n_{\text{base}} \times (1 + \sum \text{counts})$ examples: every base example is re-rendered under perturbations of six kinds, `paraphrase` (one of three alternative question wordings), `evidence_order` and `context_order` (reordered items), `irrelevant_context` (up to two unused distractor sentences), `length` (one distractor added or removed) and `schema` (the prompt re-rendered as a Slack-style thread). Every perturbation must keep the decision's comparable form, the triple (ranking, deciding factor, abstained), equal to its base and must differ in content from every other example; generation fails otherwise. A family that reserves the JSON format cannot declare a `schema` perturbation, so no group straddles the format holdout. A base example and its perturbations share a group identifier, and the 1,350 examples form 294 such groups. Consistency across a group is therefore a property the data can test, and the group is the unit of dependence that the evaluation statistics must respect (Section 5).

3.4 Splitting and the held-out format

A group is never split across train, validation and test. Assignment is a stable hash: each group's rank is the first eight bytes of the SHA-256 of the split seed (42) and the group key, read as a fraction of 2^{64}. Every release holds out the JSON input format: all 349 JSON-format examples go to the test split, and validation is drawn from the remaining formats, so early stopping never sees the out-of-distribution data the test split is meant to measure. The resulting split is 820 / 181 / 349 (recorded fractions 0.607 / 0.134 / 0.259). Three families (`ctx.budget_pressure`, `route.least_privilege` and `wsp.absent_in_active`) have no JSON-format examples and contribute nothing to the test split, so 15 families appear there, in 78 groups. A near-duplicate check at a Jaccard similarity of 0.85 or more reports 0 train–test near-duplicates for v0.0.6. The consequence for what the benchmark measures is recorded as deviation D2: test scenarios come from the same families as training, so the benchmark measures transfer across input format, not generalization to new scenarios.

Table 1: The 18 scenario families of releases v0.0.6 and v0.0.7, with the examples each contributes to the release and to the test split, and the out-of-distribution shift it reserves. Task codes: npr notification prioritization, trt tool routing, mcb mission-control briefing, ctx context prioritization, rec recommendation generation, mcr memory-conflict resolution, wsr workspace reasoning. Three families have no JSON-format examples and so contribute nothing to the test split under the format holdout. Source: Kleos-Training-Data, `docs/scenarios.md`.

Family	Task	Policy	Examples	Test	Reserved shift
`ctx.budget_pressure`	ctx	`rank_by_relevance_over_recency`	120	0	unseen entities
`ctx.relevance_over_recency`	ctx	`rank_by_relevance_over_recency`	24	8	reordered evidence
`mem.explicit_over_inferred`	mcr	`defer_to_explicit_statement`	100	35	unseen formats
`mem.reliability_over_recency`	mcr	`rank_by_reliability_over_recency`	88	32	conflicting evidence
`mem.stale_explicit_conflict`	mcr	`defer_to_explicit_statement`	64	20	conflicting evidence
`mem.unresolvable_conflict`	mcr	`resolve_or_abstain_on_support`	72	24	conflicting evidence
`brief.across_workspaces`	mcb	`respect_workspace_scope`	100	35	unseen formats
`brief.what_needs_a_decision`	mcb	`rank_by_deadline_then_evidence`	24	8	context length shift
`notif.deadline_vs_evidence`	npr	`rank_by_deadline_then_evidence`	24	8	unseen entities
`notif.tied_signals`	npr	`rank_or_abstain_when_close`	100	30	unseen formats
`rec.abstain_without_evidence`	rec	`rank_or_abstain_when_close`	18	6	unseen formats
`rec.verify_before_recommending`	rec	`verify_when_evidence_weak`	100	35	unseen formats
`route.ask_when_underspecified`	trt	`ask_when_request_ambiguous`	90	30	unseen formats
`route.least_privilege`	trt	`prefer_least_privilege_source`	100	0	unseen entities
`route.source_matches_question`	trt	`select_by_evidence_need`	88	32	unseen formats
`wsp.absent_in_active`	wsr	`ask_before_crossing_workspace`	100	0	unseen entities
`wsp.cross_workspace_request`	wsr	`respect_workspace_scope`	120	40	unseen formats
`wsp.scope_boundary`	wsr	`respect_workspace_scope`	18	6	unseen formats
Total			1,350	349	

3.5 Releases and the reasoning traces

Releases are immutable and content-hashed. The two used here are v0.0.6 (2026-09-04; content hash `3cc9a744...`) and v0.0.7 (2026-10-01; `b53afa42...`), both of 1,350 examples in the same 820 / 181 / 349 split and both built from the same catalog with split seed 42. v0.0.6 introduced the relative close-call margin (replacing an absolute margin of 0.08), split the former `ask_when_underdetermined` policy into `verify_when_evidence_weak` and `ask_when_request_ambiguous`, and added the stale-statement rule; 338 of its 1,350 targets (25%) abstain or ask. v0.0.7 keeps every example and decision of v0.0.6, and the identifiers of its JSON-format (test) examples, and adds two things to the training and validation answers only: a reasoning trace (schema 1.1) and a closing line "What decided it: ⟨label⟩." on every answer, including declined ones. Each trace follows five steps and is capped at 1,200 characters: the rule the family's policy weighs; one line per item, in prompt order, mapping the prompt's wording to the policy's values (the design document's example reads "Redpine: due in 2 days (0.33), reported (0.55), high (1.0) → 0.18"); each check the policy applied and its outcome; why the deciding factor decided it; and the conclusion, either the ranking or the failed check and the question to ask. All 1,001 training and validation answers carry one; the median trace is 560 characters long and the longest 878. The four decline labels that the test split expects thereby enter the training targets for the first time: `request_ambiguous` (45 train / 15 validation), `stale_explicit_conflict` (32 / 12), `missing_input` (25 / 5) and `insufficient_separation` (22 / 4); the fifth, `ask_before_crossing`, enters in 100 training and validation answers combined, and no test item expects it. The test split of v0.0.7 is byte-identical to v0.0.6's (SHA-256 `a4decaaf...`), and the benchmark derived from it (SHA-256 `a11ffad7...`) is the same object for every run in this paper. Earlier releases, v0.0.1 to v0.0.5, record the pipeline's maturation; v0.0.5 removed 32 examples after a human review, described next.

3.6 Quality and privacy record

The pipeline promotes examples through 14 gates, 13 of them mandatory (staging, schema and identifier integrity; four privacy gates; review presence and approval; provenance; corpus deduplication; evaluation leakage; and contract rendering; only the coverage gate can be bypassed). It applies 28 detection rules in three layers: 15 secret patterns that block, 11 personal-data patterns that redact and 2 structural heuristics that flag for review, beside a vault of registered entity literals. A private-fact classifier fails any example scoring 0.75 or more, routes scores from 0.4 to 0.75 to review and passes scores below 0.4. A sealed three-split release passes 32 verification checks. Two limits of this record matter for the paper. First, the approvals in every release to date were filled from the machine review's results; the only completed human review is v0.0.5's sample of 127 examples, of which 32 were rejected (explicit statements not honored, 3; memory conflicts decided on impact, 2; deadlines overridden, 4; ranking where the policy should have asked, 10;

Table 2: Base checkpoints. Parameter counts and revisions are as recorded in the model configurations and the experiment record. Both Logos runs load the text tower of the Ministral 3 checkpoint only. Source: `docs/experiments.md` and `configs/models/`.

Adapter	Checkpoint (Hugging Face)	Revision	Parameters	License
Ministral-8B pilot (research only)	`mistralai/Ministral-8B-Instruct-2410`	`main` (not recoverable, D7)	8.0B; 36 layers	Mistral AI Research License
KLEOS Hermes v0.0.6	`mistralai/Mistral-Nemo-Instruct-2407`	`04d8a905...`	12.2B; 40 layers	Apache-2.0
KLEOS Logos v0.0.1	`mistralai/Ministral-3-14B-Instruct-2512-BF16`, text tower	`3cea74c1...`	13,506,073,600 text (+ 438,958,080 vision, not loaded)	Apache-2.0
KLEOS Logos v0.0.2	`mistralai/Ministral-3-14B-Reasoning-2512`, text tower	`51f9210f...`	13,506,073,600 text	Apache-2.0

unjustified insufficient-separation abstentions, 13). Those categories drove the v0.0.6 policy changes, and no human review of v0.0.6 or v0.0.7 has been completed. Second, in 48 examples, 17 of them in the test split, the deciding factor is the largest raw gap between the top two items even when that gap favors the runner-up, a labelling quirk documented in the v0.0.7 design. No example in any release comes from a real user: every example was generated from the synthetic scenario catalog with fictional entities and captured through an offline mock backend, and the production-observation lane is rejected by the provenance gate.

4 Models and training

4.1 Base models

Table 2 lists the four base checkpoints, all released by Mistral AI [Mistral AI, 2024b, 2025c, Liu et al., 2026] and all loaded through Transformers 5.16.1 and PEFT 0.20.0. The pilot run used Ministral-8B-Instruct-2410 [Mistral AI, 2024a], released under the Mistral AI Research License, which confines that adapter to research; the run requested revision `main`, and the commit it resolved to is not recoverable from the preserved artifacts (deviation D7). KLEOS Hermes is built on Mistral-Nemo-Instruct-2407 [Mistral AI, 2024c]: 12.2B parameters in 40 layers with hidden size 5,120, MLP width 14,336 and a vocabulary of 131,072, under Apache-2.0, pinned to revision `04d8a905...`. KLEOS Logos is built on Ministral 3 14B, whose checkpoints ship a 13,506,073,600-parameter text model beside a 438,958,080-parameter vision encoder and projector. Both Logos runs load the text tower only (40 layers, hidden size 5,120, MLP width 16,384, 32 query and 8 key-value heads of dimension 128, an untied output head) and leave the vision weights on disk. Logos v0.0.1 used the BF16 export of the Instruct checkpoint [Mistral AI, 2025a], revision `3cea74c1...`, and v0.0.2 the Reasoning checkpoint [Mistral AI, 2025b], revision `51f9210f...`, whose chat template renders an assistant message's `reasoning` field as a `[THINK]...[/THINK]` span (token ids 34 and 35, both special) before the answer. The choice of Ministral 3 as the stronger base for H8 rested on published figures from different harnesses and years (MMLU 79.4 against Nemo's 68.0), which the record marks as indicative only.

4.2 The recipe

One QLoRA recipe [Dettmers et al., 2023, Hu et al., 2022] trained all four adapters (Table 3). The base model is frozen and stored in 4-bit NormalFloat with double quantization; the compute dtype resolves to float16 on a T4, so every run trained in fp16. LoRA adapters of rank 16 with $\alpha = 32$ and dropout 0.05 attach to the seven projection matrices of every transformer block (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`); the output head and the embeddings are excluded. That gives 252 adapted modules and 43,646,976 trainable parameters on the pilot, and 280 modules on the 40-layer models: 57,016,320 parameters for Hermes (per layer $147{,}456 + 98{,}304 + 98{,}304 + 147{,}456 + 3 \times 311{,}296 = 1{,}425{,}408$) and 60,948,480 for both Logos adapters (the wider MLP contributes $3 \times 344{,}064$ per layer). The optimizer is paged 8-bit AdamW [Dettmers et al., 2022b] at a learning rate of 2×10^{-4} with a cosine schedule, a warmup ratio of 0.03 (10 steps), no weight decay and gradient clipping at 1.0. One example per device with 8 accumulation steps gives an effective batch of 8; three epochs over 820 examples are 309 steps. Sequences are capped at 1,024 tokens, which truncates nothing: the longest training example is 440 tokens under the Hermes tokenizer, 448 after padding under Logos v0.0.1's, and 736 under v0.0.2's, where the longest reasoning

Table 3: The training recipe shared by all four runs, with the per-run differences. Source: `docs/training.md` and the training configurations under `configs/training/`.

Setting	Value
Quantization	QLoRA: NF4 with double quantization, base frozen; compute dtype `auto`, which is float16 on a T4
LoRA	$r = 16$, $\alpha = 32$, dropout 0.05, no bias, default initialization; targets `q,k,v,o,gate,up,down_proj`; `lm_head` and `embed_tokens` excluded
Adapted modules / trainable parameters	pilot 252 / 43,646,976; Hermes 280 / 57,016,320; Logos v0.0.1 and v0.0.2 280 / 60,948,480
Optimizer	paged AdamW 8-bit, learning rate 2×10^{-4}, cosine schedule, warmup ratio 0.03 (10 steps), weight decay 0, max gradient norm 1.0
Batch and epochs	1 per device × 8 accumulation steps; 3 epochs over 820 examples = 309 steps
Sequence length	1,024 tokens; nothing truncated
Loss	assistant tokens only; for Logos v0.0.2 the reasoning trace and the answer
Validation and checkpoints	every 50 steps (pilot, Hermes) or 25 (Logos); keep 3 checkpoints plus the best; ship the lowest validation loss
Precision and memory	fp16 with gradient checkpointing; Logos adds a memory probe and a gradient check before training
Seed	42 (`deterministic=True`)
Software	transformers 5.16.1, peft 0.20.0 and bitsandbytes 0.50.2 in every run; torch 2.11.0+cu128 on Colab; accelerate 1.14.0 recorded for Hermes and both Logos runs (Appendix A)

span is 298 tokens. The loss is computed on assistant tokens only; for v0.0.2 the supervised span is the trace and the answer together. Validation runs every 50 steps for the pilot and Hermes and every 25 for Logos, three checkpoints plus the best are retained, and the adapter with the lowest validation loss is the one shipped. The record describes these settings as engineering defaults, not tuned hyperparameters; no search was run. Appendix A lists the remaining settings and the software versions of each run.

Two details of the chat templates affected training. Mistral's chat template injects the system message into the last user turn; a training sequence ends on the assistant turn, so the system prompt would have been dropped from every training example. The formatter therefore folds it into the first user turn when the template drops it (deviation D5, fixed before the first run). Ministral 3's tokenizer carries a pre-tokenizer regex that Transformers flags as incorrect; the corrected regex is enabled for both Logos runs and changes the tokenization of 0 of the 1,350 examples. The remaining differences between runs were declared before each run. Logos v0.0.1 differed from Hermes in the base, tokenizer and template; the validation cadence (25 steps instead of 50); strict configuration checking; the allocator setting `expandable_segments`; protection of the best checkpoint from pruning; and a gradient check and a memory probe at the configured batch size before the trainer re-seeds. Logos v0.0.2 differed from v0.0.1 in the base and the data; the generation budget at evaluation (1,024 new tokens instead of 512); the hardware (two T4s, model parallel); adapter selection on a loss that covers the trace as well as the answer; and the results schema.

4.3 Compute, runs and memory

The pilot trained on a Colab Tesla T4 (compute capability 7.5, fp16); Hermes and Logos v0.0.1 trained and evaluated on one Colab Tesla T4 (14.56 GB, compute capability 7.5, CUDA 12.8); and Logos v0.0.2 trained on Kaggle's two T4s with the model's layers spread across both by `device_map: auto` (GPU 0 held the embeddings and layers 0 to 8; GPU 1 held layers 9 to 39, the final norm and the output head). The Colab runs used torch 2.11.0+cu128; Kaggle's torch version is recorded only in the run's private environment file. In these records "GB" means 2^{30} bytes. Measured peak memory was 9.67 GB for the pilot, 13.09 GB for Hermes, 13.70 GB for Logos v0.0.1 and 10.21 GB over the whole run for Logos v0.0.2; the memory probe before that run's first step peaked at 10.16 GB on GPU 1 and 4.75 GB on GPU 0, against estimates of 8.53 and 6.38 GB: the total matched, the split did not (finding L-F6). The first memory estimator was about 5 GB too optimistic for Hermes (7.96 to 8.12 GB predicted against 13.09 GB measured, finding H-F10); the rebuilt estimator reproduces the pilot and Hermes peaks within 0.1% and the Logos v0.0.1 probe within 1.3%. Findings of this kind, 25 in all across the three registers, are listed in Appendix E.

Table 4 summarizes the runs. Hermes trained in two sessions: the Colab runtime died after step 250 while writing a

Table 4: The four training runs. "Val" is validation loss; for Logos v0.0.2 it covers the trace and the answer. The pilot's `train_runtime` of 1,970.8 s is in its audit but is not reconciled with the 25.8 to 27.5 s per step of the later T4 runs, so no duration is cited for it; the Logos v0.0.1 record keeps per-session timings only. Sources: the artifact audit and the three run reports under `docs/experiments/`.

Run	Dates (trained)	Hardware	Sessions	Trainer time	Best checkpoint (val)	Final val	Peak memory
Ministral-8B pilot	2026-09-09 (frozen 2026-09-15)	1 × T4, Colab	not recorded	not reconciled	step 200, epoch 1.95 (0.03987)	0.04469	9.67 GB
KLEOS Hermes v0.0.6	2026-09-17 to 09-18	1 × T4, Colab	2; resumed from step 250 after 18.03 h	11,396 s incl. final validation	step 200, epoch 1.95 (0.04343)	0.04515	13.09 GB
KLEOS Logos v0.0.1	2026-09-24 to 09-27	1 × T4, Colab	4; resumed from steps 100 and 250	per session only; session 4: 2,157.6 s for 59 steps	step 175, epoch 1.70 (0.03824)	0.04969	13.70 GB
KLEOS Logos v0.0.2	2026-10-06	2 × T4, Kaggle, model parallel	1	15,951 s (4.43 h)	step 175, epoch 1.70 (0.0293)	0.0311	10.21 GB (run); 10.16 GB (GPU 1 probe)

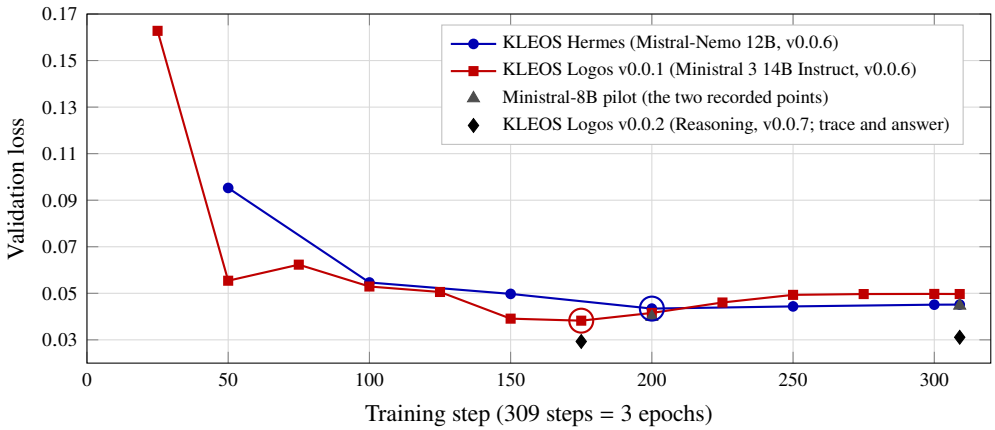


Figure 2: Validation loss by training step. Circles mark the checkpoints that were selected on validation loss and shipped: step 200 (epoch 1.95) for Hermes, 0.04343, and step 175 (epoch 1.70) for Logos v0.0.1, 0.03824. Both recorded curves rise through the third epoch, and in the other two runs the final value exceeds the selected one. The pilot and Logos v0.0.2 records keep only the best and final values; v0.0.2's loss covers the reasoning trace as well as the answer. The tokenizers differ between models, so the curves compare in shape, not in level.

checkpoint, and training resumed eighteen hours later on a different T4 instance from the last complete checkpoint (deviation D8). Logos v0.0.1 needed four sessions and two resumes within the free tier's usage limits (deviation D9). Logos v0.0.2 trained in one Kaggle session. Trainer time for Hermes was 11,222 s over the two sessions plus 174 s for the final validation pass, 3 h 09 m 56 s in all, at 25.8 to 27.5 s per step, with nine validation passes totalling 1,645 s (about 187 s each in the first session and 174 s in the second). Logos v0.0.1's record keeps per-session timings but no single total. Logos v0.0.2's trainer reported 15,951 s (4.43 h), about 38.5 s per optimizer step and 51.6 s per step including validation, in a notebook that ran 17,746 s with installation, download and a smoke run. The pilot's recorded trainer time is not reconciled with the later runs and is not cited.

4.4 Checkpoint selection and resume verification

Figure 2 shows the validation curves. In the two runs with a recorded curve, Hermes and Logos v0.0.1, the validation loss reaches its minimum before the end of the second epoch and rises through the third; the pilot and Logos v0.0.2 record only their best and final values, and in both the final value is above the selected one. The pilot and Hermes select step 200 (epoch 1.95), Logos v0.0.1 and v0.0.2 select step 175 (epoch 1.70). The final training loss sits about 10 times below the validation loss for Hermes and about 15 times below it for Logos v0.0.1, which is consistent with a third epoch that memorizes the 820 training targets; selection on validation loss keeps that epoch out of the shipped adapter. Two

Table 5: The evaluation arms and the runs in which each was evaluated. Source: `docs/evaluation.md` and the README.

Arm	Model	Prompting	Evaluated in
`arm0_base`	base	none	no (a 20-example probe only)
`arm1_base_orchestrated`	base	KLEOS orchestration	pilot, Hermes, Logos v0.0.1
`arm2_finetuned`	base + LoRA adapter	controlled task context	all four runs
`arm3_finetuned_orchestrated`	base + LoRA adapter	KLEOS orchestration	no

consequences were declared in advance. Logos' 25-step validation grid could select step 175 where Hermes' 50-step grid could not, and for Logos v0.0.2 the selection criterion covers the trace, which is 58% of the validation targets by characters, so no answer-only validation loss exists for it. The optimizer skipped 4 steps on fp16 overflow during the Hermes run (through step 305) and 2 during Logos v0.0.1 (both before step 250); the pilot and v0.0.2 records do not report the count.

The interrupted runs were verified rather than assumed faithful. For Hermes, the gradient-check loss before training was bit-identical in both sessions (2.1058197021484375), the scheduler state carried over, and the two independent evaluations of step 300, one per session, agree to 1.4×10^{-5} (0.045107 against 0.045121). For Logos v0.0.1, the first resume point was checked by size (368 MB, a complete checkpoint) and the second file by file (weights, optimizer, scheduler, random-number and scaler state), and the final validation pass on the loaded best weights reproduced the recorded loss exactly. For Hermes and Logos v0.0.1 the exported adapter is byte-identical to the selected checkpoint's weights (SHA-256 `dc121fa3...` and `f3e8dcdc...`); for the pilot and Logos v0.0.2 the selected checkpoint is established from the recorded validation loss and the retained checkpoints, and v0.0.2's shipped adapter is `e49724f6...` (Appendix F). Each run's manifest records the hash of its full configuration, the git commit it ran at, the hash of the loaded dataset files and whether any configuration value was adjusted at runtime (none was).

5 Evaluation protocol

5.1 Benchmark, arms and what was run

The benchmark is derived from the sealed test split: 349 items over the seven tasks, in 78 groups, of which 271 are answerable (the reference commits) and 78 should-decline (the reference declines), in 61 and 17 groups respectively. Every item is in the JSON input format, which no training example uses. The protocol defines four arms (Table 5); the pre-registered research question names the prompt-engineered orchestration baseline, so the comparisons in this paper are `arm1_base_orchestrated` against `arm2_finetuned` (deviation D1: H1 had named the bare `arm0_base`, of which only a 20-example probe exists, scored 0.5659, a biased head-of-file slice that is not comparable with a full run). If that probe were representative, the registered comparison would show a smaller gain than the orchestrated one does; the full bare-base run that would settle this has not been made. The orchestrated arm `arm3` was never run, so the interaction hypothesis H7 is only partly measured. Logos v0.0.2 was evaluated on `arm2` only: H9 does not ask whether fine-tuning helps, H8a had answered that for the architecture, and a base arm would have cost about 7 GPU hours (estimated) of Kaggle's 30-hour weekly quota. Fine-tuning's own effect on v0.0.2 is therefore not measured.

5.2 Decoding and the reasoning split

Every arm decodes greedily (temperature 0, top-p 1, no sampling, repetition penalty 1) with batch size 1 and one seed, since greedy decoding makes the seed irrelevant. The generation budget is 512 new tokens, raised to 1,024 for Logos v0.0.2 because the worst case computed from the data, twice the longest training trace plus the longest benchmark answer, is $2 \times 298 + 199 = 795$ tokens. For the Reasoning model the completion is split on token id 35 before decoding and only the answer is graded; a completion whose trace never closes has an empty answer, is graded as such, and is marked as cut. The model card for that checkpoint recommends sampling at temperature 1; greedy decoding was kept so that every arm shares one decoding rule and so that the served model can be checked against the evaluation byte for byte (Section 8). One consequence, finding L-F7, is that the stop reason reads "stop" whenever the trace closed even if the answer was then cut by the budget; 6 of the 349 v0.0.2 completions were cut that way, all after a closed trace, and were scored as generated.

5.3 The grader

The composite grader `kleos_policy` scores three components against the reference. The *ranking* component is $0.6 \times \text{nDCG} + 0.4 \times \text{top-1}$ accuracy against the reference ranking [Järvelin and Kekäläinen, 2002]; nDCG was bounded after finding F1 so that a repeated item cannot score above a perfect ranking. The *deciding factor* is read from the `deciding_factor` key of a JSON answer, from a "What decided it:" line, or from the first allowed label the answer mentions, and compared with the reference label. *Confidence* is read from an explicit boolean in a JSON answer or, otherwise, from a list of abstention phrasings taken from the KLEOS corpus, and compared with whether the reference commits. The score is the mean of the components the reference supports. Whether the response is a JSON object (`format_valid`) is reported as a sub-score and excluded from the score (deviation D6): under the format holdout a format failure must be distinguishable from a judgment failure. Kendall's τ [Kendall, 1938] and nDCG are reported as sub-scores. Three faithfulness heuristics (evidence coverage, citation precision and an unsupported-claim rate) are text-level heuristics, not entailment checks; the fabricated-citation heuristic flags 47.9% of the gold test answers (finding H-F12) and evidence coverage is vacuous because no benchmark item names decisive evidence identifiers (H-F13), so they appear in this paper only beside those floors. Consistency across a group is measured by the agreement rate (the group reached one decision) and the correct-agreement rate (it agreed and was right). The configured grouping unit for the first runs was the scenario family, under which a perfect model scores only 0.600 because 6 of the 15 test families mix different correct labels (finding H-F11); from H8 on, consistency is also reported by the perturbation group, where a perfect model scores 1.000.

5.4 Subsets and the unlearnable labels

Deviation D3 was discovered during the analysis of the pilot: 78 of the 349 test items, the 78 should-decline items, expect a deciding-factor label that appears zero times in the v0.0.6 training targets (`request_ambiguous` 30, `missing_input` 25, `stale_explicit_conflict` 20, `insufficient_separation` 3). The cause is a rendering rule: in v0.0.6, decline labels were printed only in JSON-format answers, and the format holdout put every JSON-format example in the test split, so 0 of the 124 training declines carry a label and 492 of the 820 training answers (60%) have no "What decided it:" line at all. For a model trained on v0.0.6 the deciding-factor component is therefore capped at 0.7765 and the score at 0.9255 on the full benchmark, and the deciding factor is read on the answerable items as well as overall (0.816 on the 271 answerable items for the pilot, 0.6332 overall). This affects both arms of every v0.0.6 comparison identically, so H1 stays fair, but it is the reason the two cross-model hypotheses, H8b and H9, name the 271 answerable items as their primary population, with the 78 should-decline items as the first secondary row. v0.0.7 trains the four decline labels for the first time, and the should-decline row is the one it was built to move; H9 kept the answerable subset as primary on the stated ground that switching after the fact would "pick the population most likely to flatter the result."

5.5 Statistics

Differences are paired per item. A difference's 95% interval comes from 2,000 bootstrap resamples with seed 42, taking the resampled differences at positions $\lfloor 0.025B \rfloor$ and $\lfloor 0.975B \rfloor$ of the sorted list; the two-sided p-value is twice the fraction of resamples on the far side of zero. When no resample crossed, the comparison tool prints $p = 0.0$ (H1) or $p < 0.0005$ (H8 onward, which is $1/B$); under the doubled one-sided fraction the two-sided bound is $2/B$, so this paper reports every such case as $p < 0.001$. A protocol amendment of 2026-09-24, adopted before any Logos training, changed the resampling unit: from H8 on, every interval resamples the 78 groups rather than the 349 items, because perturbations of one case are not independent and item-level intervals are too narrow (finding H-F14; Field and Welsh, 2007, Miller, 2024). Item-level intervals are still printed beside the cluster ones, no earlier result was re-decided, and intervals over fewer than 5 groups are reported as not estimable, which affects the 8-item `context_prioritization` task. With 61 answerable and 17 should-decline groups, cluster inference is itself approximate; with few clusters it tends to over-reject [Cameron et al., 2008]. Per-task rows are reported without correction for multiple comparisons and are not decisive. The decision rules are fixed in advance. For a fine-tuning hypothesis (H1, H8a) a task improves only if its interval excludes zero, and the hypothesis is supported if at least one task improves and none regresses. For a cross-model hypothesis (H8b, H9) the verdict on the paired cluster interval of the primary difference is *better* if the interval lies entirely above zero, *worse* if entirely below, *equivalent* if entirely within ± 0.02, and *inconclusive* otherwise. Every result in this paper is reported as an estimate with its interval; a null is never read as equivalence [Altman and Bland, 1995].

Table 6: The pre-registered hypotheses. "Registered" is the date and post-rewrite commit of the entry; H5 to H7 carry no decision rule in the record. The record reports no interval for H3, although its rule names one. Source: `docs/experiments.md`, Status table and hypothesis entries.

Id	Question	Primary metric	Decision rule	Registered	Status
H1	Fine-tuning beats the prompt-engineered baseline	per-task grader score on the held-out test split (registered as arm0 vs arm2; run as arm1 vs arm2, D1)	improvement counts only if the paired bootstrap 95% CI excludes zero; per task, no blended aggregate	2026-08-14, `fd87931`	Supported on the pilot (2026-09-15, with D1 and D2); replicated by Hermes (2026-09-22)
H2	Gains generalize out of distribution	`ood_score` beside `in_distribution_score`; generalization gap	gains generalize if the OOD delta CI excludes zero	2026-08-14	Not measurable: the v0.0.6 benchmark is entirely out of distribution (2026-09-15)
H3	Decisions survive irrelevant perturbations	`correct_agreement_rate`; flips per perturbation kind	consistency improves if the delta CI excludes zero	2026-08-14	Improved, still poor (pilot 2026-09-15; reproduced exactly by Hermes 2026-09-22); no interval computed
H4	No loss of general capability	`relative_delta` on a fixed capability suite	regression if < -0.01 with a CI excluding zero	2026-08-14	Not run
H5	The effect depends on scale	—	—	2026-08-14; re-scoped 2026-09-15	Not run; re-scoped to two Mistral models
H6	Transfer across model families	—	—	2026-08-14; closed 2026-09-15	Closed: not testable with one model family
H7	Fine-tuning and orchestration interact	—	—	2026-08-14	Partially measured: arm1 and arm2 only, in three runs (2026-10-01)
H8a	Fine-tuning helps Logos v0.0.1	per-task `kleos_policy`, Logos arm1 vs arm2	a task improves only if its cluster CI excludes zero; supported if at least one improves and none regresses	2026-09-24, `81e3e9a`	Supported (2026-10-01)
H8b	Logos v0.0.1 is better than Hermes (primary)	mean `kleos_policy` on the 271 answerable items, paired	better / worse / equivalent (± 0.02) / inconclusive, by the cluster CI of Logos − Hermes	2026-09-24, `81e3e9a`	Inconclusive (2026-09-29)
H9	Logos v0.0.2, trained to think, is better than Hermes (primary)	as H8b	as H8b	2026-10-06, `6d7e450`	Supported: better, +0.0409, cluster 95% CI +0.0040 to +0.0780 (2026-10-06)

5.6 Pre-registration

Table 6 lists the nine hypotheses with their metrics, decision rules, registration dates and outcomes. H1 to H7 and the fixed protocol (held-out splits only, greedy decoding identical across arms, a benchmark and graders fixed before the run, per-task reporting with no blended aggregate, recorded seeds, a leakage check before training) were committed on 2026-08-14 with the repository's first commit. H5 was re-scoped and H6 closed on 2026-09-15, after the pilot and before Hermes was trained. H8 and the clustered-interval amendment were committed on 2026-09-24, before any Logos training; deviation D11 was declared on 2026-09-29 before the arm it concerns ran; H9 was committed on 2026-10-06, before any Logos v0.0.2 training. In each case the commit that adds the entry is the registration and each run's manifest records the commit it ran at, so the order of registration and run is checkable from the public history (commit identifiers here are the post-rewrite ones; Appendix F maps them to the identifiers the frozen records cite). Thirteen deviations, D1 to D13, are dated and logged with their effect on the result (Appendix D).

6 Results

Every result in this section is pre-registered unless marked as a re-report or as descriptive. Scores are means of the `kleos_policy` grader on the 349-item benchmark. Intervals are 95% bootstrap intervals, item-level for the H1 comparisons and cluster-level by perturbation group from H8 on, as the protocol fixes. Table 7 collects the main numbers, Table 8 the per-task results, Table 9 the comparisons between fine-tuned models and Table 10 the grader sub-scores; Figures 3 to 5 draw them.

6.1 H1: the pilot and its replication

The pilot run on Ministral-8B compared the orchestrated base against the fine-tuned adapter on all 349 items. Its first report scored the baseline at 0.5231. An artifact audit then found that nDCG could exceed 1.0 on a ranking that repeats an item (finding F1), and 60% of the baseline's answers repeated one. Bounding nDCG re-graded 210 of the 349

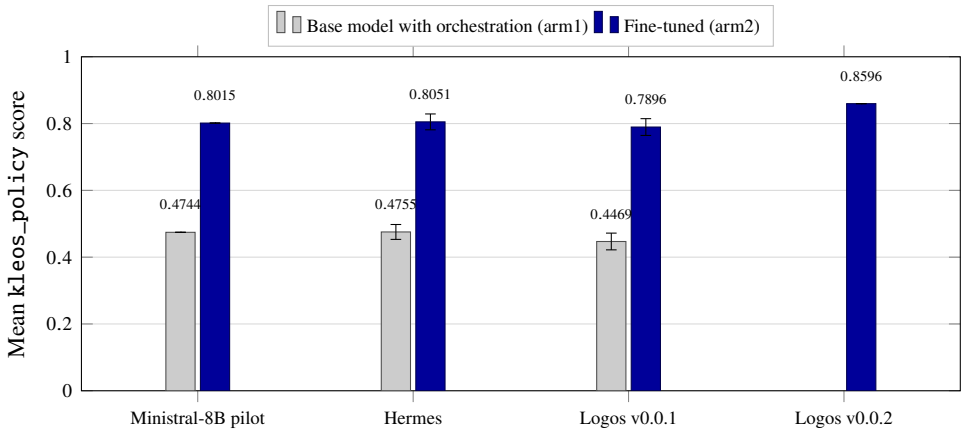


Figure 3: Mean `kleos_policy` score on the 349-item benchmark. Error bars are the recorded item-level 95% bootstrap intervals (Hermes and Logos v0.0.1 only; the pilot baseline was re-graded after its interval was recorded, and the Logos v0.0.2 overall mean is descriptive). Logos v0.0.2 has no base arm. Item-level intervals treat the 349 items as independent and are narrower than the cluster intervals used from H8 on (Section 5).

Table 7: Main results on the 349-item benchmark. Base and fine-tuned intervals are item-level; the answerable and should-decline columns are re-reports by group with cluster intervals where recorded. The pilot's baseline was re-graded after its interval was recorded, so neither pilot figure carries one; Logos v0.0.2 has no base arm and its overall mean is descriptive. Consistency is the agreement rate by perturbation group, where a perfect model scores 1.000. Sources: `docs/experiments.md` (H1, H8, H9 entries) and the run reports.

Model	Base, arm1	Fine-tuned, arm2	Difference	Answerable (271 items, 61 groups), arm2	Should-decline (78 items, 17 groups), arm2	Consistency, arm2 (all / answerable)
Ministral-8B pilot	0.4744	0.8015	+0.3271; 7 of 7 tasks $p < 0.05$	not reported	not reported	not reported
KLEOS Hermes	0.4755 [0.4532, 0.4978]	0.8051 [0.7813, 0.8288]	+0.3295 [0.3068, 0.3521], $p < 0.001$	0.8976 [0.8634, 0.9301] (base 0.5570)	0.4835 (base 0.1923)	0.769 / 0.770 (base 0.346)
KLEOS Logos v0.0.1	0.4469 [0.4219, 0.4720]	0.7896 [0.7644, 0.8147]	+0.3427 (descriptive); answerable +0.3952 [+0.3443, +0.4438]	0.8808 [0.8381, 0.9191] (base 0.4857 [0.4445, 0.5261])	0.4724 (base 0.3123; +0.1601 [+0.0712, +0.2409])	0.833 / 0.853 (base 0.154 / 0.164)
KLEOS Logos v0.0.2	not run	0.8596 (descriptive)	—	0.9385 [0.9083, 0.9663]	0.5855	0.833 / 0.869

baseline responses and none of the fine-tuned ones, moved the baseline to 0.4744, left the fine-tuned score at 0.8015, and raised the count of tasks with a significant improvement from six to seven. The corrected gap is +0.3271, with every task improved at $p < 0.05$ under the pre-registered item-level bootstrap (Table 8, first block); parse failures fell from 13 to 0 (descriptive).

KLEOS Hermes replicated the result on a different base one week later: 0.4755 [0.4532, 0.4978] to 0.8051 [0.7813, 0.8288], a paired difference of +0.3295 [0.3068, 0.3521]. The comparison tool prints $p = 0.0$, meaning that none of the 2,000 resamples crossed zero, so the correct statement is $p < 0.001$. Every task improved with $p < 0.001$ (Table 8, second block; Figure 4), from +0.1904 on notification prioritization to +0.4323 on workspace reasoning; the 8-item context-prioritization task moved from 0.5935 to 1.0000 (interval 0.2612 to 0.5352). The sub-scores (Table 10) show where the gain comes from: the ranking component rose from 0.5985 to 0.8822 (nDCG 0.6174 to 0.9527, top-1 accuracy 0.5702 to 0.7765, Kendall's τ 0.5449 to 0.8479), the deciding factor from 0.0860 to 0.6132 and confidence from 0.7421 to 0.9198. `format_valid` is 0.0000 in both arms with 0 parse failures: neither arm produced the held-out JSON format, and the scores measure judgment on prose answers. Descriptively, fine-tuning also shortened the answers, from a mean of 226 completion tokens to 94 (medians 217 and 91); one baseline answer hit the 512-token cap and no fine-tuned answer did; mean latency on the T4 fell from 24.7 s to 16.1 s. Re-reported by group after the H8 amendment, Hermes scores 0.8976 [0.8634, 0.9301] on the 271 answerable items (base 0.5570) and 0.4835 on the 78 should-decline items (base 0.1923), and its agreement rate by group is 0.769 (base 0.346).

The two fine-tuned models land within 0.004 of each other overall (0.8015 and 0.8051) and the two baselines within 0.002; the per-task differences between the adapters range from −0.0766 on notification prioritization to +0.1076 on

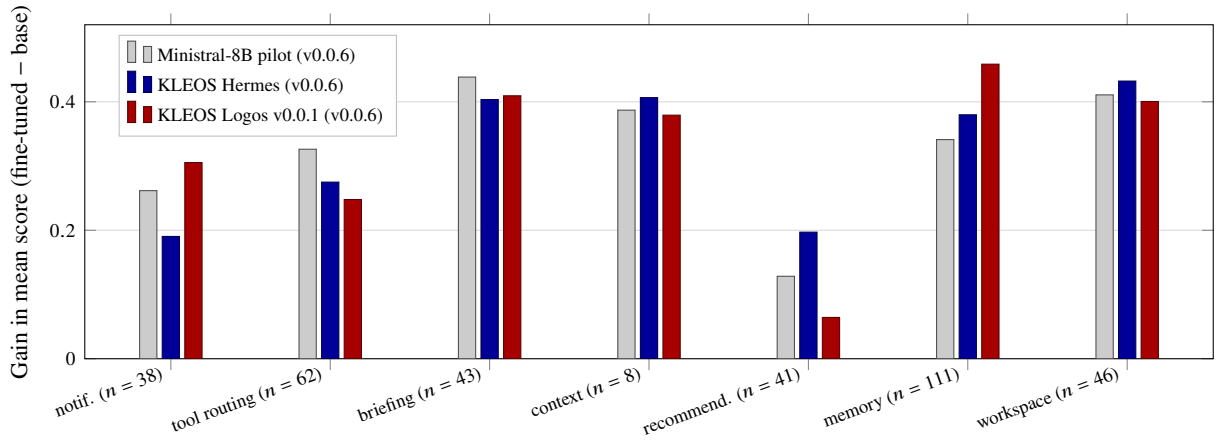


Figure 4: Per-task gain from fine-tuning on the 349-item benchmark, by task (test items per task in parentheses). Every pilot and Hermes gain was significant under the pre-registered item-level bootstrap ($p < 0.05$ and $p < 0.001$ respectively); for Logos v0.0.1, five tasks improved under the cluster bootstrap, `recommendation_generation` did not reach significance, and `context_prioritization` (8 items, fewer than 5 groups) was not estimable by groups. Per-task intervals are not recorded, so no error bars are drawn (Table 8).

context prioritization. The record treats these as leads, not results: scale, architecture and pretraining are confounded between the two bases, and the agreement is cross-base consistency, not a replication of one configuration across seeds.

6.2 H8: a stronger base

H8a (supported). Fine-tuning helped the Ministral 3 base as it had helped the others: Logos v0.0.1 moved from 0.4469 [0.4219, 0.4720] to 0.7896 [0.7644, 0.8147] overall. Under the cluster bootstrap five tasks improved with $p < 0.001$ and none regressed (Table 8, third block); recommendation generation (+0.0643) was not significant, and the 8-item context-prioritization task was not estimable by group. On the answerable subset the paired cluster difference is +0.3952 [+0.3443, +0.4438] (0.4857 to 0.8808) and on the should-decline subset +0.1601 [+0.0712, +0.2409] (0.3123 to 0.4724); agreement by group rose from 0.154 to 0.833. One caveat is recorded beside these gains: 41 of the base's 349 answers (11.7%) were cut at the 512-token budget, against none of the adapter's (mean completion length 348 against 95 tokens), which may inflate the base-to-adapter differences.

H8b (inconclusive). On the primary population Hermes scores 0.8976 [0.8634, 0.9301] and Logos v0.0.1 0.8808 [0.8381, 0.9191]; the paired cluster difference over 61 groups is −0.0168 [−0.0546, +0.0177] (Table 9, Figure 5). The interval crosses zero and is not contained in the ±0.02 equivalence margin, so the pre-registered verdict is inconclusive: a Hermes lead of up to 0.0546 is not ruled out, nor is a Logos lead of up to 0.0177. The secondary rows point the same way. On the should-decline subset the difference is −0.0111 [−0.0429, +0.0195]. Per task, recommendation generation regressed from 0.4946 to 0.3906 (−0.1040; $p = 0.006$ by group, $p < 0.001$ by item) and workspace reasoning by −0.0142 ($p = 0.014$ by group, not significant by item); notification prioritization rose by +0.0837 ($p = 0.007$ by item, not significant by group); the other three estimable tasks' group intervals included zero (per-task intervals are not recorded). Agreement by group was higher for Logos (0.833 against 0.769; 0.853 against 0.770 on answerable groups), with no interval computed (descriptive). The overall means, 0.8051 and 0.7896, are descriptive.

6.3 H9: a Logos trained to think

H9 (supported: better). Logos v0.0.2 scores 0.9385 [0.9083, 0.9663] on the answerable items against Hermes' 0.8976 [0.8634, 0.9301]; the paired cluster difference over 61 groups is +0.0409 [+0.0040, +0.0780], $p = 0.031$. The interval lies entirely above zero, so the pre-registered verdict is better; the record notes that the lower bound is close to zero. The first secondary row, the one the v0.0.7 data was built to move, did not reach significance by group: on the should-decline items Logos v0.0.2 scores 0.5855 against 0.4835, +0.1020 [−0.0018, +0.2062], $p = 0.054$ over 17 groups. By task, notification prioritization (+0.1324, $p < 0.001$), recommendation generation (+0.0893, $p = 0.002$) and

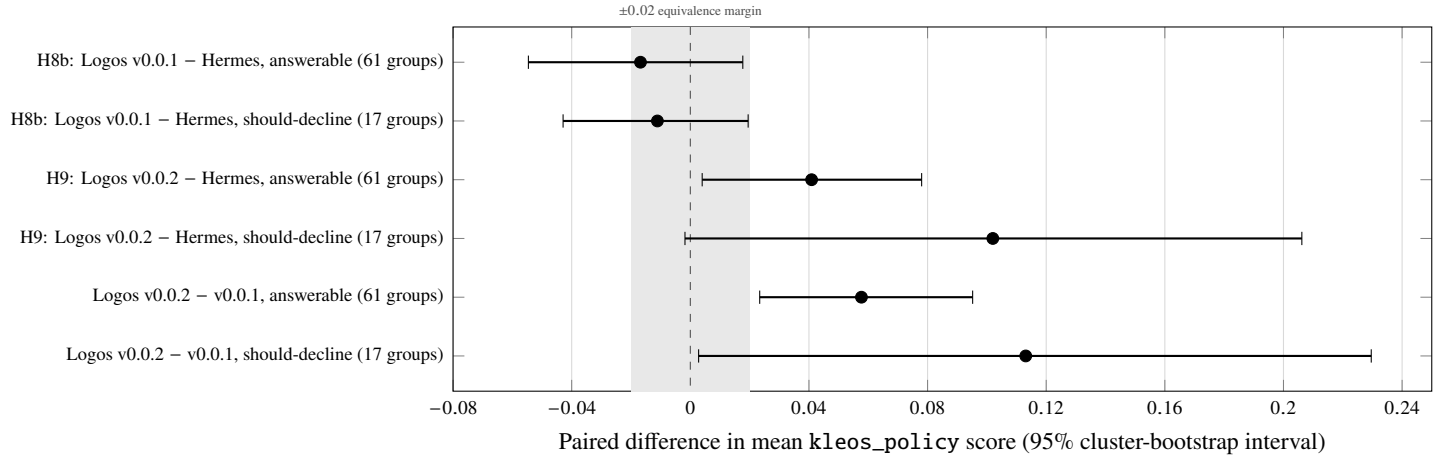


Figure 5: Paired differences between fine-tuned models, resampled by benchmark group (2,000 resamples). The primary rows are the answerable ones: H8b is inconclusive because its interval crosses zero and is not contained in the ±0.02 equivalence margin, and H9 is "better" because its interval lies entirely above zero. The should-decline rows rest on 17 groups; only the v0.0.1 to v0.0.2 row excludes zero.

memory-conflict resolution (+0.0737, $p = 0.048$) improved; tool routing (+0.0330), workspace reasoning (+0.0172) and mission-control briefing (−0.0153) had group intervals that included zero; context prioritization (−0.0002) was not estimable. Descriptively, agreement by group rose from 0.769 to 0.833 over all groups and from 0.770 to 0.869 over the answerable ones. The overall mean, 0.8596, is descriptive: v0.0.2 has no base arm.

Logos v0.0.1 to v0.0.2 (secondary). Against its own predecessor, v0.0.2 gains +0.0577 [+0.0234, +0.0952] on the answerable items ($p < 0.001$) and +0.1131 [+0.0028, +0.2296] on the should-decline items ($p = 0.046$); recommendation generation (+0.1933, $p < 0.001$) and memory-conflict resolution (+0.0941, $p = 0.005$) improved and no task regressed. Among the comparisons between fine-tuned models, this is the one in which the should-decline difference excludes zero, and it confounds the same three changes as H9.

The traces (descriptive). All 349 v0.0.2 completions carried a reasoning trace and every trace closed; the traces average 549 characters (median 540, maximum 840) against 575 in the validation targets. Completions averaged 293.5 tokens (median 285, 95th percentile 413) and took 53.8 s on average on the two T4s (95th percentile 76.3 s). Six completions reached the 1,024-token budget after a closed trace, all on answerable memory-conflict items, where the answer repeated a phrase; they were scored as generated, at 0.59 to 0.65 each. `format_valid` is 0.0 for all three models.

Reported, not decisive. The secondary rows in this section are uncorrected for multiple comparisons. Three sit near the conventional threshold: memory-conflict resolution under H9 ($p = 0.048$ by group), the H9 should-decline row ($p = 0.054$) and the v0.0.1 to v0.0.2 should-decline row ($p = 0.046$). None of them bears on the pre-registered verdicts, which rest on the primary rows alone.

7 Analysis

Everything in this section is exploratory: it reads the recorded sub-scores, consistency and abstention tables for what the models learned, and no claim here was registered in advance.

Where the gain comes from. The sub-scores in Table 10 decompose the H1 gain. For Hermes the deciding-factor component moved most, from 0.0860 to 0.6132: the base model rarely named the factor the policy weighs, the adapter usually does. The ranking component moved from 0.5985 to 0.8822, with nDCG (0.6174 to 0.9527) rising more than top-1 accuracy (0.5702 to 0.7765), one reading is that the adapter orders the tail of a ranking better than it picks its head. Confidence, the component that scores whether the answer commits or declines when the reference does, rose from 0.7421 to 0.9198. Two ceilings bound these figures for every model trained on v0.0.6: because the 78 should-decline

Table 8: Per-task results for the three runs with a base arm. Verdicts for the pilot and Hermes are under the pre-registered item-level bootstrap; verdicts for Logos v0.0.1 are under the cluster bootstrap by group (the item-level verdict is given where it differs). Per-task intervals are not recorded except the one shown. Sources: `docs/experiments.md` H1 and H8a tables; Hermes run report.

Task	n	Base	Fine-tuned	Difference	Verdict
Ministral-8B pilot, corrected after F1 (2026-09-15)					
mission-control briefing	43	0.5323	0.9708	+0.4385	improved, $p < 0.05$
workspace reasoning	46	0.5228	0.9336	+0.4108	improved, $p < 0.05$
context prioritization	8	0.5054	0.8924	+0.3870	improved, $p < 0.05$
memory-conflict resolution	111	0.4781	0.8192	+0.3412	improved, $p < 0.05$
tool routing	62	0.3919	0.7181	+0.3262	improved, $p < 0.05$
notification prioritization	38	0.6253	0.8871	+0.2617	improved, $p < 0.05$
recommendation generation	41	0.3285	0.4569	+0.1284	improved, $p < 0.05$
overall	349	0.4744	0.8015	+0.3271	7 of 7 tasks improved
KLEOS Hermes (2026-09-22)					
workspace reasoning	46	0.5190	0.9512	+0.4323	improved, $p < 0.001$
context prioritization	8	0.5935	1.0000	+0.4065	improved, $p < 0.001$; interval 0.2612 to 0.5352
mission-control briefing	43	0.5681	0.9716	+0.4035	improved, $p < 0.001$
memory-conflict resolution	111	0.4756	0.8553	+0.3798	improved, $p < 0.001$
tool routing	62	0.3929	0.6679	+0.2750	improved, $p < 0.001$
recommendation generation	41	0.2974	0.4946	+0.1971	improved, $p < 0.001$
notification prioritization	38	0.6201	0.8105	+0.1904	improved, $p < 0.001$
overall	349	0.4755	0.8051	+0.3295	interval 0.3068 to 0.3521, $p < 0.001$
KLEOS Logos v0.0.1, H8a (2026-10-01), verdicts by group					
memory-conflict resolution	111	0.3763	0.8350	+0.4586	improved, $p < 0.001$
mission-control briefing	43	0.5552	0.9646	+0.4094	improved, $p < 0.001$
workspace reasoning	46	0.5366	0.9370	+0.4005	improved, $p < 0.001$
context prioritization	8	0.6204	0.9995	+0.3792	not estimable by group (8 items); improved by item, $p < 0.001$
notification prioritization	38	0.5888	0.8942	+0.3054	improved, $p < 0.001$
tool routing	62	0.4021	0.6501	+0.2479	improved, $p < 0.001$
recommendation generation	41	0.3263	0.3906	+0.0643	not significant
overall	349	0.4469	0.7896	+0.3427	descriptive; 5 tasks improved, none regressed: supported

items expect labels absent from the training targets (deviation D3), the deciding factor cannot exceed 0.7765 and the score cannot exceed 0.9255 on the full benchmark. On the 271 answerable items the fine-tuned deciding factor is 0.7897 for Hermes (214 correct) and 0.816 for the pilot (221 correct), against 0.1107 (30 correct) for the Hermes base. Logos v0.0.1 shows the same shape with a slightly weaker deciding factor (0.5989) and ranking (0.8529).

Consistency. Table 11 reports how often a model gives the same decision across the items of one group. Under the configured unit, the scenario family, the fine-tuned Hermes agrees with itself on 0.333 of the 15 test families and is right when it agrees on 0.333, against 0.133 and 0.000 for its base; the pilot reproduces both fine-tuned figures exactly. Those numbers sit under a ceiling of 0.600, because 6 of the 15 families mix different correct labels, which is why the record re-reports consistency by perturbation group, where a perfect model scores 1.000. Hermes rises from 0.346 to 0.769 by group, Logos v0.0.1 from 0.154 to 0.833, and Logos v0.0.2 reaches 0.833 over all groups and 0.869 over the answerable ones. Fine-tuning reduced flips under every perturbation kind for Hermes (paraphrase 53 to 44, evidence order 36 to 21, irrelevant context 24 to 21, length 5 to 3, context order 1 to 0), but paraphrase remains the largest source of inconsistency in both arms. H3's verdict, "improved, still poor," summarizes this: two thirds of the test families (10 of 15) still flip under a perturbation that should not change the decision.

Abstention was learned by scenario family. The lower block of Table 11 splits the should-decline items by the family that produces them. Two families decline unconditionally in training (every one of their 32 and 45 training targets abstains), and on their 50 test items the fine-tuned Hermes declines 50 times out of 50, as does the pilot. Two families decline conditionally, on the evidence (25 of 55 and 3 of 9 training targets abstain), and on their 28 test items both adapters decline 0 times. On the 271 answerable items both adapters commit 271 times. The record's reading is that the models learned "this kind of question → decline," not "the evidence is insufficient → decline." The base model has the opposite failure: it over-commits almost everywhere, declining correctly on only 3 of the 78 should-decline items. Logos v0.0.2 raised the should-decline score to 0.5855 after training on targets that include the four decline labels, but whether it declines on the evidence rather than on the family was not analyzed.

Table 9: Comparisons between fine-tuned models. Every interval is a paired cluster-bootstrap 95% interval by group; per-task rows give the difference and the verdict by group, with the item-level verdict where it differs. Consistency is the agreement rate by group. Per-task intervals are not recorded; "n.s." is the recorded verdict that the group interval included zero. Sources: `docs/experiments.md` H8b and H9 results; Logos v0.0.1 and v0.0.2 run reports.

Row	H8b: Logos v0.0.1 − Hermes	H9: Logos v0.0.2 − Hermes	Logos v0.0.2 − v0.0.1 (secondary)
Answerable, 271 items, 61 groups (primary)	−0.0168 [−0.0546, +0.0177]; inconclusive	+0.0409 [+0.0040, +0.0780], $p = 0.031$; better	+0.0577 [+0.0234, +0.0952], $p < 0.001$
Should-decline, 78 items, 17 groups	−0.0111 [−0.0429, +0.0195]	+0.1020 [−0.0018, +0.2062], $p = 0.054$; not significant	+0.1131 [+0.0028, +0.2296], $p = 0.046$
Per task, by group	recommendation −0.1040 regressed ($p = 0.006$); workspace −0.0142 regressed ($p = 0.014$; n.s. by item); notification +0.0837 n.s. (improved by item, $p = 0.007$); briefing −0.0070, tool routing −0.0178, memory −0.0204 n.s.; context not estimable	notification +0.1324 ($p < 0.001$), recommendation +0.0893 ($p = 0.002$), memory +0.0737 ($p = 0.048$) improved; tool routing +0.0330, workspace +0.0172, briefing −0.0153 n.s.; context −0.0002 not estimable	recommendation +0.1933 ($p < 0.001$), memory +0.0941 ($p = 0.005$) improved; no task regressed
Consistency by group, all / answerable	0.769 / 0.770 → 0.833 / 0.853	0.769 / 0.770 → 0.833 / 0.869	0.833 / 0.853 → 0.833 / 0.869
Overall mean (descriptive)	0.8051 vs 0.7896	0.8051 vs 0.8596	0.7896 vs 0.8596

Table 10: Grader sub-scores on the full benchmark. The score is the mean of the ranking, deciding-factor and confidence components where the reference supports them; `format_valid` is reported and excluded. Sources: Hermes and Logos v0.0.1 run reports.

Sub-score	Hermes base	Hermes fine-tuned	Logos v0.0.1 base	Logos v0.0.1 fine-tuned
score (`judgment`)	0.4755	0.8051	0.4469	0.7896
ranking	0.5985	0.8822	0.4468	0.8529
nDCG	0.6174	0.9527	0.4505	0.9459
top-1 accuracy	0.5702	0.7765	0.4413	0.7135
Kendall's τ	0.5449	0.8479	0.5350	0.8345
deciding factor	0.0860	0.6132	0.1805	0.5989
confidence	0.7421	0.9198	0.7135	0.9169
`format_valid`	0.0000	0.0000	0.0000	0.0000
parse failures	0	0	0	0

Selection, memorization and the trace. Section 4 showed that in the two runs with recorded curves the validation loss turns upward in the third epoch while the training loss keeps falling, to about a tenth (Hermes) or a fifteenth (Logos v0.0.1) of the validation loss; the other two runs record only that their final validation loss exceeds the selected checkpoint's. The shipped adapters are the ones selected before that turn, at epochs 1.95 and 1.70, and the benchmark results above belong to those checkpoints, not to the end of training. For Logos v0.0.2 the selection criterion is a loss over the trace and the answer together, and the trace is 58% of the validation targets by characters; the record keeps no answer-only loss, so it is not known whether the step that minimized the joint loss also minimized the answer loss.

What H9 does and does not say. Logos v0.0.2 differs from Hermes in three ways at once: the base checkpoint (Ministral 3 14B Reasoning instead of Mistral-Nemo 12B), the training data (v0.0.7, with reasoning traces, instead of v0.0.6) and the generation budget (1,024 instead of 512 tokens). The pre-registration counted the first two as the confounded changes and listed the thinking budget as a further one, and the hardware and the torch build differ as well. Hermes was not retrained on v0.0.7, and v0.0.2 has no base arm. H9 therefore says that the new model is measurably better than the old on the answerable items, with a lower bound of +0.0040 that the record calls close to zero, and it does not say which change produced the gain or how large each part is. Within the Logos line, v0.0.2 against v0.0.1 isolates the data and budget changes from the base change only partly, since the base changed too (Instruct to Reasoning). The experiment that would separate them, Hermes retrained on v0.0.7 beside a v0.0.2 base arm, is the obvious next run; neither has been made.

Heuristics reported beside their floors. The faithfulness heuristics moved in the expected direction, and the record declines to make anything of it. For Hermes, citation precision rose from 0.5938 to 0.7192 and the unsupported-claim rate fell from 0.3400 to 0.1600; the number of responses flagged as containing a fabricated citation fell from 147 to 98 (194 to 116 for the pilot, 216 to 94 for Logos v0.0.1). The same heuristic flags 167 of the 349 gold test answers (47.9%),

Table 11: Consistency and abstention. Upper block: agreement across the items of one group, by the configured unit (15 scenario families, where a perfect model scores 0.600) and by perturbation group (78 groups, perfect score 1.000); flips are counts of decision changes attributed to each perturbation kind. Lower block: correct declines per abstention family, and correct commitments on the answerable items. "Unspecified" is the record's label for flips not attributed to a named perturbation kind; the record has no row for the schema kind. Sources: Hermes run report; `docs/experiments.md` H3.

Measure	Hermes base	Hermes fine-tuned	Pilot fine-tuned
Agreement rate by family (ceiling 0.600)	0.133	0.333	0.333
Correct-agreement rate by family	0.000	0.333	0.333
Families flipping under some perturbation (of 15)	13	10	10
Flips: paraphrase / unspecified / evidence order / irrelevant context / length / context order	53 / 39 / 36 / 24 / 5 / 1	44 / 28 / 21 / 21 / 3 / 0	not reported
Agreement rate by group (ceiling 1.000)	0.346	0.769	not reported
`mem.stale_explicit_conflict` (training: 32 of 32 decline), 20 test items	0 / 20	20 / 20	20 / 20
`route.ask_when_underspecified` (45 of 45 decline), 30 test items	0 / 30	30 / 30	30 / 30
`rec.verify_before_recommending` (25 of 55 decline, conditional), 25 test items	3 / 25	0 / 25	0 / 25
`rec.abstain_without_evidence` (3 of 9 decline, conditional), 3 test items	0 / 3	0 / 3	0 / 3
Should commit, 271 test items	256 / 271	271 / 271	271 / 271

so a flag is not evidence of a fabrication, and evidence coverage reads 1.0000 for every arm because no benchmark item names decisive evidence identifiers. These numbers are kept in the record as uncalibrated heuristics and are not findings.

Generation economy. Fine-tuning made the answers shorter and faster without any instruction to do so: mean completion length fell from 226 to 94 tokens for Hermes and from 348 to 95 for Logos v0.0.1, and mean latency on the T4 from 24.7 to 16.1 s and from 36.7 to 17.2 s, consistent with the adapters having learned the terse format of the targets. The reasoning model reverses part of that: Logos v0.0.2's completions average 293.5 tokens, of which the trace is the larger part, and 53.8 s on the two-GPU evaluation host; Section 8 gives the served latencies.

8 Serving as a product component

Packages and pins. Each served model is a deployment package: the adapter weights, the tokenizer files and a manifest that pins the base checkpoint to a commit and records the adapter's SHA-256 (`dc121fa3...` for Hermes, the frozen step-200 weights; `e49724f6...` for Logos v0.0.2, the step-175 weights). The research copy of Hermes' adapter configuration recorded no revision (finding H-F1); the package copy pins `04d8a905...`, and every later adapter records its revision at training time. Packages live in private model repositories and are read by the serving host at startup. The serving code loads the base in NF4 with float16 compute, applies the adapter and decodes greedily with the evaluated budget (512 tokens for Hermes, 1,024 for Logos v0.0.2), so that the served model is the evaluated model and its outputs can be checked against the frozen evaluation byte for byte. The reply carries the answer (`text`) and, for Logos v0.0.2 under the version-2 contract, the trace (`reasoning`), which the KLEOS application shows under "How Logos decided."

The free GPU host. Both models run on Hugging Face Spaces ZeroGPU [Hugging Face, 2026], which allocates half of an NVIDIA RTX Pro 6000 Blackwell (48 GB, compute capability 12.0) for the duration of a call. A free account has 5 minutes of GPU time per day, charged to the calling account and reset 24 hours after that account's first use; a call is admitted only while 1.5 times its requested duration remains, and it waits up to 60 s for a GPU. Each call requests

$$\min\big(\max(\lceil 10 + \text{max_new_tokens}/12 \rceil,\ 15),\ 60\big) \text{ seconds}, \tag{2}$$

with the constants exposed as Space variables: 53 s for Hermes and 60 s for Logos v0.0.2 (the rule gives 96 s at 1,024 tokens, capped at 60), so admission needs 80 s and 90 s of remaining quota respectively. The record estimates roughly 20 to 30 Hermes answers a day and about 9 Logos v0.0.2 answers per fresh daily window; raising its cap to request the full budget's worth of time would cut that to about 5 (all three estimated). A Logos v0.0.2 answer that loops runs out of its 60 s, is aborted and is reported to the application as a model error, which the application treats like a cut answer.

Table 12: Serving measurements on ZeroGPU. Hermes figures are medians over 10 calls on 2026-09-23 with answers of 69 to 128 tokens; Logos figures are from the two-day smoke test of 2026-10-07. "Estimated" rows follow the admission rule, not a measurement. Source: `docs/experiments/serving-verification-records.md`.

Measure	KLEOS Hermes v0.0.6	KLEOS Logos v0.0.2
Generation speed	about 12 tokens/s, cold and warm	11.0 tokens/s warm, 10.9 cold
Warm call, median	7.9 s (0.15 s to obtain a GPU)	19.7 s (19.4 s generating)
Cold call, median	12.9 s (4.1 s to obtain a GPU); slowest 34 s	29.9 s (5.5 s to obtain a GPU)
GPU seconds per answer	7 to 13	24 to 26 (217.8 to 232.3 for 9 answers)
Peak GPU memory	8.34 GiB	8.86 GiB
Startup: package download / base load and quantization	2 s / 41 s (363 weights)	1.9 s / 57.1 s
Reproduction of the frozen evaluation	9 of 9 (Colab T4) and 9 of 9 (ZeroGPU), byte for byte	8 of 9 over two quota windows; prompt tokens 9 of 9
Requested GPU duration / quota needed for admission	53 s / 80 s	60 s / 90 s
Answers per day on a free account (estimated)	20 to 30	about 9 per fresh window

Reproduction checks. Hermes was checked twice on 2026-09-23. On a Colab T4 the package reproduced 9 of 9 frozen evaluation outputs byte for byte (one prompt from each of the seven tasks plus two should-decline items); the record calls this a reproducibility result, not a score. On ZeroGPU, on different hardware, the same 9 of 9 reproduced byte for byte, 9 of 9 prompt token counts matched and a repeated call returned an identical answer. Logos v0.0.2 was deployed on 2026-10-07 and checked over two daily quota windows, because the first day's window, already partly used before the test, admitted only 7 of the 9 prompts: on day 1 the quota admitted 7 of the 9 prompts, of which 6 reproduced exactly, with prompt tokens equal on all 7; on day 2 the remaining 2 reproduced exactly. In total 8 of 9 answers and traces are identical, prompt tokens match on 9 of 9 and the decisions agree on 8 of 9 (Table 12). The divergent item is an answerable briefing case. Its trace is identical for 568 of its 761 characters; then one sentence goes another way ("Startup has. . . " where the evaluation wrote "Startup is the active workspace. . . "). The answer declines and asks before looking in another workspace (label `ask_before_crossing`), where the evaluation had ranked the three items in the active workspace (label `scope`, scored 1.0). The record's diagnosis is GPU arithmetic, not setup: the same weights, prompt and decoding rule on a different GPU with float16 rounding. The recorded decision (2026-10-07) was to accept and document: Logos v0.0.2 is served as a Beta, and claims about it say that it measured better than Hermes on the benchmark, not that the live Space reproduces that score. Re-measuring the full benchmark on the Space would take over five weeks at about 9 answers a day (estimated). Appendix G keeps the host facts and the day-by-day smoke record.

Container. A Docker image for Hermes was built (6.31 GB, `linux/amd64`) from an allowlisted build context of 57 files, and 257 CPU-side tests passed inside it; it has not been run on a GPU host and is not part of the verified serving path.

9 Limitations

One training run per configuration. Each adapter was trained once, with seed 42, because of compute limits. The intervals in this paper therefore cover the variability of sampling evaluation items and groups; they do not cover training variance from initialization and data order, which is substantial in fine-tuning [Dodge et al., 2020]. Differences between configurations are conditional on one run each and may not replicate across seeds. The agreement between the pilot and Hermes (within 0.004 overall) is consistency across two bases, not a replication of one configuration.

The data and the benchmark are generated by the same policy. Answers, traces and references are all computed from the policies described in Section 3, so the benchmark measures how well a model learned those policies, not whether the policies are good or whether the model reasons in general. Test scenarios come from the same families as the training scenarios, with only the input format held out (deviation D2), so the results speak to transfer across format,

not to new scenarios. No completed human review exists for v0.0.6 or v0.0.7; the only calibration evidence for the synthetic labels is v0.0.5's review, which rejected 32 of 127 sampled examples before the policy changes that v0.0.6 made, and 48 examples carry a known labelling quirk.

Few clusters and uncorrected rows. The cluster intervals rest on 61 answerable and 17 should-decline groups, and cluster inference with few clusters tends to over-reject. The 8-item context-prioritization task is not estimable by group. Per-task rows are uncorrected for multiple comparisons, and H1's per-task verdicts were never recomputed by group; the protocol amendment left them under their original item-level rule.

No model produced the held-out format. `format_valid` is 0.0 for every model and arm. Format is excluded from the score by design, so the results measure judgment on prose answers given JSON prompts; whether a model can also produce the JSON structure the product's parsers would prefer is unmeasured.

Abstention is a family shortcut, and the decline labels were unlearnable for three of the four runs. Section 7 shows that the v0.0.6 adapters decline by scenario family, not by evidence, and whether Logos v0.0.2 does better was not analyzed. For the v0.0.6 runs, 78 of 349 items expect labels absent from training (deviation D3), which caps their scores and is why the cross-model comparisons use the answerable subset. Beyond the decline labels, two families are labelled `impact` in the test split and only `evidence` in training, so 94 of the 349 test items (26.9%) need a family–label pairing that training never shows.

Arms not run. The bare base model and the fine-tuned-plus-orchestration arm were never evaluated in full, so H7 is only partly measured, and the comparisons use the orchestrated baseline rather than the registered bare one (deviation D1). Logos v0.0.2 has no base arm, so the effect of fine-tuning on the Reasoning checkpoint is not measured; H9 compares two fine-tuned models only.

H9 confounds three changes. The base checkpoint, the training data and the generation budget changed together, the hardware and torch build differ, and Hermes was not retrained on v0.0.7. The interval's lower bound, +0.0040, is close to zero and rests on 61 clusters, where cluster inference tends to over-reject; the verdict is one run, one seed, one benchmark.

Decoding, selection and the base's budget. The Reasoning model was decoded greedily although its card recommends sampling at temperature 1; 6 of 349 completions looped and were scored as generated. Its adapter was selected on a loss that covers the trace, and no answer-only loss exists. For Logos v0.0.1, 41 of the base's answers were cut at 512 tokens, which may inflate that run's base-to-adapter gains. Validation losses are not comparable across tokenizers, so the curves in Figure 2 compare in shape only.

Serving, provenance and tuning. The served GPU differs from the evaluated one; Logos reproduced 8 of 9 checks and its live outputs have not been re-measured on the benchmark. The pilot's base revision is not recoverable, so that run is reproducible only modulo upstream not having moved; the Kaggle image was not pinned (deviation D12) and its torch version is recorded only privately. The hyperparameters are engineering defaults, untuned. The faithfulness measures are text heuristics, not entailment checks. A 24B vision-language configuration in the repository was never trained.

9.1 Data provenance and licensing

Every training and test example is synthetic, generated from a catalog of fictional entities and captured through an offline mock backend; no release contains conversations, records or other data from real users, and the pipeline's provenance gate rejects any example from a production-observation lane. The two production bases, Mistral-Nemo-Instruct-2407 and Ministral-3-14B-Reasoning-2512, are released under Apache-2.0; the pilot's base, Ministral-8B-Instruct-2410, is under the Mistral AI Research License, which is why that adapter is research-only and was never served. The trained adapters are distributed privately through the deployment packages rather than published; the training and evaluation code, the frozen records and the data pipeline are public under MIT. The misuse surface of a model that ranks a student's own items and declines on weak evidence is narrow, and the serving endpoints require a key and run under a per-account quota.

10 Reproducibility statement

Anyone can run the training and evaluation harness and its 1,638 collected tests without a GPU, evaluate the pipeline end to end on the public development fixtures with an echo backend, and check every configuration hash in this paper except the pilot's against the committed configurations. Anyone can rebuild the data: at the recorded commit of the companion repository, the public pipeline reproduces release v0.0.6 byte for byte (content hash `3cc9a744...`) and, from the reasoning change on, release v0.0.7 (`b53afa42...`); the test split and the benchmark then carry the SHA-256s cited in Section 3. Holders of a release can retrain and re-evaluate each adapter from the runbooks in the repository with the recorded configurations, commits and seeds, and compare their adapters against the recorded SHA-256s and their results against the frozen result-file hashes (Appendix F). Nobody can reproduce the pilot's base revision, which was never recorded (deviation D7); its run is reproducible only if the upstream `main` pointer has not moved. Every run's manifest records its configuration hash, git commit, dataset hash and runtime adjustments (none), and each verdict in this paper is traceable to a dated entry in the research record. The repository's history was rewritten before publication to remove assistant co-author trailers from seven commit messages; the content, authors and dates of the commits are unchanged, and a table in the record maps each identifier cited by the frozen records to its public counterpart, six of which appear in Appendix F.

11 Conclusion

A small open-weight model can learn to reproduce a product's decision policy on its benchmark from 820 synthetic examples on a free GPU. The learning is large, visible in every task and consistent across three base checkpoints: fine-tuning moved Hermes from 0.4755 to 0.8051 on a benchmark whose answers are computed by the policy, with a 95% interval of 0.3068 to 0.3521 on the difference, replicating the pilot, and it moved Logos v0.0.1 by a similar amount. A stronger base alone did not produce a measurably better model; the Reasoning checkpoint of that base, trained on data that adds policy-derived reasoning traces, did, by +0.0409 on the answerable items with an interval that excludes zero but starts close to it. What the models learned is specific: they order and label the way the policy does, they decline by the kind of question rather than by the evidence, and they never produce the held-out format. Each model is one run, the second comparison confounds three changes, and the benchmark measures transfer across format within the scenarios the policy was written for. The contribution this paper claims is therefore not a method but an account: a pre-registered, fully logged record of what a student can measure about fine-tuning with the compute a student has, in which the per-task regressions, the inconclusive result and the measurably better one are reported to the same standard.

Acknowledgements

Mistral AI released the open-weight base models. Hugging Face maintains Transformers, PEFT and bitsandbytes and hosts the models on Spaces ZeroGPU. Google Colab and Kaggle provided the free GPU sessions on which every run was trained and evaluated.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, pages 29304–29320, 2021.

Douglas G. Altman and J. Martin Bland. Statistics notes: Absence of evidence is not evidence of absence. *BMJ*, 311 (7003):485, 1995. doi: 10.1136/bmj.311.7003.485.

Luca Bertinetto, João F. Henriques, Samuel Albanie, Michela Paganini, and Gül Varol. Preface. In *NeurIPS 2020 Workshop on Pre-registration in Machine Learning*, volume 148 of *Proceedings of Machine Learning Research*, page i. PMLR, 2021. URL https://proceedings.mlr.press/v148/bertinetto21a.html.

A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008. doi: 10.1162/rest.90.3.414.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. GPT3.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 30318–30332, 2022a. doi: 10.52202/068431-2198. arXiv:2208.07339, titled "LLM.int8()".

Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization. In *International Conference on Learning Representations (ICLR 2022)*, 2022b. URL https://openreview.net/forum?id=shpkpVXzo3h. arXiv:2110.02861.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 10088–10115, 2023. doi: 10.52202/075280-0441. arXiv:2305.14314.

Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. arXiv preprint arXiv:2002.06305, 2020.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-1128.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Number 57 in Monographs on Statistics and Applied Probability. Chapman & Hall, 1993. ISBN 0-412-04231-2. doi: 10.1201/9780429246593.

C. A. Field and A. H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14852–14882. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.830.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-acl.507.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR 2022)*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9. arXiv:2106.09685.

Hugging Face. Spaces ZeroGPU: dynamic GPU allocation for Spaces. Hugging Face Hub documentation. https://huggingface.co/docs/hub/spaces-zerogpu, 2026. Accessed 2026-10-13.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.

M. G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1–2):81–93, 1938. doi: 10.1093/biomet/30.1-2.81.

Zachary C. Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship. *Queue*, 17(1):45–77, 2019. doi: 10.1145/3317287.3328534.

Alexander H. Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, et al. Ministral 3. arXiv preprint arXiv:2601.08584, 2026.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv preprint arXiv:2411.00640, 2024.

Mistral AI. Ministral-8B-Instruct-2410 (model card). Hugging Face. https://huggingface.co/mistralai/Ministral-8B-Instruct-2410, 2024a. Mistral AI Research License.

Mistral AI. Mistral NeMo. Blog post, July 18, 2024. https://mistral.ai/news/mistral-nemo, July 2024b.

Mistral AI. Mistral-Nemo-Instruct-2407 (model card). Hugging Face. https://huggingface.co/mistralai/Mistral-Nemo-Instruct-2407, 2024c. Revision `04d8a90549d23fc6bd7f642064003592df51e9b3`, as pinned

in the KLEOS Models repository.

Mistral AI. Ministral-3-14B-Instruct-2512-BF16 (model card). Hugging Face. https://huggingface.co/mistralai/Ministral-3-14B-Instruct-2512-BF16, 2025a. Revision 3cea74c1ebaf5ce5f5a2553de470e2ceab825142, as pinned in the KLEOS Models repository.

Mistral AI. Ministral-3-14B-Reasoning-2512 (model card). Hugging Face. https://huggingface.co/mistralai/Ministral-3-14B-Reasoning-2512, 2025b. Revision 51f9210f3cd20f3452a80d5819d15dc61cc50630, as pinned in the KLEOS Models repository.

Mistral AI. Introducing Mistral 3. Blog post, December 2, 2025. https://mistral.ai/news/mistral-3, December 2025c.

Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* 2019)*, pages 220–229. ACM, 2019. doi: 10.1145/3287560.3287596.

Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. GitHub repository. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Emiel van Miltenburg, Chris van der Lee, and Emiel Krahmer. Preregistering NLP research. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 613–623. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.naacl-main.51.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.754.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837, 2022. doi: 10.52202/068431-1800. arXiv:2201.11903.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-demos.6.

A Hyperparameters and software

Table 13 lists the settings that Table 3 omits and the software versions of each run.

Table 13: Training settings not shown in Table 3, and the software versions per run. "not recorded" means the public record keeps no value. Sources: `docs/training.md`, the training configurations under `configs/training/` and the run reports.

Setting	Value
Logging interval	every 5 steps (all four configurations)
Validation and checkpoint interval	50 steps (pilot, Hermes); 25 steps (Logos v0.0.1, v0.0.2)
Checkpoints retained	3, plus the best (`load_best_model_at_end` on the lowest validation loss)
Strict configuration checking	off (pilot, Hermes); on (Logos v0.0.1, v0.0.2)
PEFT options	`use_dora` false, `use_rslora` false, `modules_to_save` none, default LoRA initialization (recorded for the pilot and Hermes; not inspected for Logos v0.0.1)
Warmup	ratio 0.03, resolved by Transformers 5 to 10 steps
Mistral tokenizer regex fix	off for Hermes; on for both Logos runs (changes 0 of 1,350 examples)
Adapter files (fp32)	pilot 174,655,536 bytes; Hermes 228,140,600; Logos v0.0.1 243,869,280; Logos v0.0.2 not recorded
Trainable share reported by PEFT	pilot 1.5293%; Hermes 1.38%
fp16-skipped optimizer steps	Hermes 4 (through step 305); Logos v0.0.1 2 (both before step 250); pilot and v0.0.2 not recorded
Pilot software	transformers 5.16.1, peft 0.20.0, bitsandbytes 0.50.2, torch 2.11.0+cu128, Python 3.13.15
Hermes software	torch 2.11.0+cu128, transformers 5.16.1, peft 0.20.0, accelerate 1.14.0, bitsandbytes 0.50.2, datasets 4.8.5
Logos v0.0.1 software	as Hermes, plus tokenizers 0.23.2
Logos v0.0.2 software	transformers 5.16.1, peft 0.20.0, accelerate 1.14.0, bitsandbytes 0.50.2, tokenizers 0.23.2 installed over Kaggle's image; torch is Kaggle's own, recorded only in the run's environment file
Serving (Logos Space)	torch 2.11.0+cu128, transformers 5.16.1, peft 0.20.0, bitsandbytes 0.50.2, gradio 6.28.0, spaces 0.51.3; the Hermes Space pins pydantic 2.12.5

B Prompt and answer format

An example is a conversation of three messages. The sealed releases are not published as files (the public pipeline rebuilds them byte for byte), so the format is shown here on the first public development fixture shipped with the training harness, which its metadata marks as a synthetic development fixture and not the research dataset. It has the same structure as a release example without a reasoning trace.

```
system:  You are the reasoning layer of KLEOS. Deterministic systems have already
         retrieved and ranked the context. Decide, then justify from the evidence
         provided. Do not invent facts.

user:    Rank these items by what deserves attention first.

         - internship-application: deadline closes in 2 days; evidence strong
           (confirmed opening)
         - networking-event: deadline in 3 weeks; evidence weak (no confirmed
           attendees)
         - resume-review: deadline no deadline; evidence medium (peer feedback
           offered)

assistant:
         1. internship-application
         2. resume-review
         3. networking-event
```

```
Reasoning: internship-application comes first because its deadline is
the nearest (closes in 2 days) and the evidence of impact is strong
(confirmed opening). Items with weak evidence or no deadline rank last,
since acting on them displaces work with a firm cost of delay.
```

In release v0.0.7 every training and validation answer ends with a line "What decided it: ⟨label⟩." and carries a `reasoning` field. For the Reasoning base the chat template renders that field before the answer, so a supervised completion is `[THINK]`trace`[/THINK]`answer`</s>`, with `[THINK]` as token 34 and `[/THINK]` as token 35, and the loss covers the trace and the answer. The trace follows five steps:

1. the rule the family's policy weighs, for example "Score each item as deadline × evidence × impact";
2. one line per item, in prompt order, mapping the prompt's wording to the policy's values, for example "Redpine: due in 2 days (0.33), reported (0.55), high (1.0) → 0.18";
3. each check the policy applied and its outcome: close call (relative separation below 0.15), stale explicit statement (30 days or older), usable evidence (weight 0.55 or above), workspace scope, strong challenger (weight 0.8 or above) and ambiguous request;
4. why the deciding factor decided it;
5. the conclusion: the ranking for a committed answer, or the failed check and the question to ask for a declined one.

Traces are capped at 1,200 characters and follow wording rules so that they pass the privacy and fact checks: only item names are capitalized and never two capitalized words in a row, numbers use at most two decimal places, and no dates, at-signs or comma-grouped numbers appear. Each step has two or three phrasings, chosen deterministically, so a rebuild reproduces every trace.

C Policies, labels and the test population

Table 14 reproduces the twelve policies and Table 15 the eleven deciding-factor labels with the policy that produces each; the scenario families are in Table 1.

Table 14: The twelve decision policies: what each orders by, when it declines, and the deciding factor it names. Copied from the policy table of Kleos-Training-Data, `docs/scenarios.md`.

Policy	Orders by	Abstains when	Deciding factor
`rank_by_deadline_then_evidence`	Score	Never	Deadline, evidence or impact, whichever differs most between the top two
`rank_or_abstain_when_close`	Score	The top two are within a 15% relative separation	`insufficient_separation`
`rank_by_reliability_over_recency`	Evidence, then recency	Never	Evidence, or deadline when evidence ties
`resolve_or_abstain_on_support`	Evidence, then recency	The top two are within 15% on evidence and were recorded at the same time	`insufficient_separation`
`select_by_evidence_need`	Evidence × impact, then freshness	Never	Evidence or impact
`rank_by_relevance_over_recency`	Impact (relevance), then recency	Never	Impact, or deadline when impact ties
`respect_workspace_scope`	In-scope items first, then score	Never. Out-of-scope items are flagged, not acted on.	`scope` when scopes are mixed, otherwise the score ranking
`ask_before_crossing_workspace`	Same as `respect_workspace_scope`	No in-scope item is adequate and the strongest out-of-scope item is	`ask_before_crossing`
`defer_to_explicit_statement`	Explicit statements, then evidence, then recency	The explicit statement is at least 30 days old and a newer inferred record has evidence of at least 0.8	`explicit_statement`, `evidence` or `stale_explicit_conflict`
`verify_when_evidence_weak`	Score	No item is adequate	`missing_input`
`ask_when_request_ambiguous`	Score	The family sets `prompt.request_ambiguous`	`request_ambiguous`
`prefer_least_privilege_source`	Adequacy, then narrowest reach, then evidence	Never	`least_privilege` when several sources are adequate, otherwise `evidence`

Table 15: The eleven deciding-factor labels named by the policy table, the policy-table cell that produces each (quoted verbatim, with the policy named), and what the record says about the test split. The benchmark's label set has nine options; the two labels whose only families have no JSON-format examples are absent from it. Test counts are recorded for the four decline labels only. Source: Kleos-Training-Data, `docs/scenarios.md`, policy table; `docs/experiments.md`, deviation D3; the v0.0.7 repair specification.

Label	Kind	Policy-table cell	Test split
`deadline`	commit	`rank_by_deadline_then_evidence`: "Deadline, evidence or impact, whichever differs most between the top two"; `rank_by_reliability_over_recency`: "Evidence, or deadline when evidence ties"; `rank_by_relevance_over_recency`: "Impact, or deadline when impact ties"	count not recorded
`evidence`	commit	the three cells above; `select_by_evidence_need`: "Evidence or impact"; `prefer_least_privilege_source`: "`least_privilege` when several sources are adequate, otherwise `evidence`"; `defer_to_explicit_statement`: "`explicit_statement`, `evidence` or `stale_explicit_conflict`"	count not recorded
`impact`	commit	`rank_by_deadline_then_evidence` and `select_by_evidence_need` (cells above); `rank_by_relevance_over_recency`: "Impact, or deadline when impact ties"	count not recorded
`scope`	commit	`respect_workspace_scope`: "`scope` when scopes are mixed, otherwise the score ranking"	count not recorded
`explicit_statement`	commit	`defer_to_explicit_statement`: "`explicit_statement`, `evidence` or `stale_explicit_conflict`"	count not recorded
`least_privilege`	commit	`prefer_least_privilege_source`: "`least_privilege` when several sources are adequate, otherwise `evidence`"	absent: its family has no JSON-format examples
`insufficient_separation`	decline	`rank_or_abstain_when_close` abstains when "The top two are within a 15% relative separation"; `resolve_or_abstain_on_support` when "The top two are within 15% on evidence and were recorded at the same time"	3 items
`ask_before_crossing`	decline	`ask_before_crossing_workspace` abstains when "No in-scope item is adequate and the strongest out-of-scope item is"	absent: its family has no JSON-format examples
`stale_explicit_conflict`	decline	`defer_to_explicit_statement` abstains when "The explicit statement is at least 30 days old and a newer inferred record has evidence of at least 0.8"	20 items
`missing_input`	decline	`verify_when_evidence_weak` abstains when "No item is adequate"	25 items
`request_ambiguous`	decline	`ask_when_request_ambiguous` abstains when "The family sets `prompt.request_ambiguous`"	30 items

D Deviations D1 to D13

Table 16 condenses the deviations log; the full text of each entry is in the research record.

Table 16: The deviations log, condensed. Each row's last column is the effect recorded at the time. Source: `docs/experiments.md`, deviations log.

Id	Date	Deviation	Recorded effect
D1	2026-09-15	H1 registered `arm0_base` vs `arm2`; the run compared `arm1_base_orchestrated` vs `arm2`	The research question names the orchestrated baseline; arm0 exists only as a 20-example probe (0.5659), a biased head-of-file slice not comparable with a full run
D2	2026-09-15	H1 registered an entity holdout; the sealed release uses a format holdout on JSON	The test split probes format transfer, not entity generalization
D3	2026-09-15	78 of 349 test items (22%) expect a label that appears zero times in training (`request_ambiguous` 30, `missing_input` 25, `stale_explicit_conflict` 20, `insufficient_separation` 3)	Affects both arms identically, so H1 stays fair; absolute deciding-factor figures carry the caveat (0.6332 overall, 0.816 on answerable items for the pilot)
D4	2026-09-15	The ranking extractor was corrected before the run (it had returned whole prose lines)	Instrument changed before any result; no reported number affected
D5	2026-09-15	The formatter folds the system prompt into the first user turn when the template drops it	Fixed before the run; without it every example would have trained without its policy instructions
D6	2026-09-15	The composite `kleos_policy` grader was added, with `format_valid` reported separately and excluded	Without the separation a format failure is indistinguishable from a judgment failure
D7	2026-09-15	The pilot used `revision: main`; the resolved commit is not recoverable	Future runs are pinned; the historical record is not backfilled
D8	2026-09-18	Hermes training was interrupted after step 250 and resumed 18 hours later on another T4	No effect: the selected checkpoint (step 200) predates the interruption; resume state verified (step-300 evaluations agree to 1.4×10^{-5})
D9	2026-09-27	Logos v0.0.1 trained over four Colab sessions with two resumes (from steps 100 and 250)	No effect on the selected adapter (step 175, written in session 2); the reported `train_loss` is a resume artifact and is not quoted
D10	2026-09-29	Logos `arm2` was evaluated in three sessions; the first two never reached storage (finding L-F3); the third generated all 349 in one pass	No effect: a single uninterrupted run at the registered commit
D11	2026-09-29 (declared before it ran)	Logos `arm1` would run at a later commit containing the L-F3 file-handling fix	Done as declared on 2026-10-01; the change touches file handling only; the second session resumed 172 recorded generations
D12	2026-10-06	Both Kaggle notebooks ran on the "Latest Container Image," not a pinned one	No effect: neither run was resumed in a later version; libraries were installed at pinned versions
D13	2026-10-06	The training run is Version 2 of its notebook; Version 1 stopped at the commit guard in its first cell	No effect: Version 1 ran no code beyond the guard; both runs used the registration commit and the configuration hash matched before training

E Findings

Table 17 lists the three findings registers.

Table 17: The findings registers, one line each, with status as of 2026-10-07. Sources: the artifact audit, the Hermes run report and `docs/experiments/logos-findings.md`.

Id	Finding	Status
F1	nDCG could exceed 1.0 on degenerate rankings; the pilot baseline re-graded from 0.5231 to 0.4744 (210 of 349 responses changed), the fine-tuned arm unchanged	resolved 2026-09-15
F2	A 17.1 MB `tokenizer.json` was refused by the 5 MB scan cap	resolved 2026-09-15
F3	The base-model revision was not pinned	resolved for future runs 2026-09-15; the pilot is not backfilled (D7)
H-F1	The research adapter configuration records `revision: null`	research copy kept; the package pins the revision; later adapters record it
H-F2	The manifest contradicts itself on the trainable count (1,342,592,000 against 57,016,320)	open; the per-layer reconciliation gives 57,016,320
H-F3	A stale sliding-window note was frozen into the records	open
H-F4	A checkpoint-saved event is not a durability guarantee on Drive	open; weightless checkpoints are rejected at resume
H-F5	The comparison tool checked benchmark identity by path, not content	fixed for later runs
H-F6	The comparability check never compares base models	open (methodological)
H-F7	Result files drop the grader details	fixed for later runs
H-F8	`train_loss` is invalid after a resume: the reported value is a resume artifact; the mean over the second session's steps 251 to 309 is 0.00459	open; never quoted
H-F9	The retention pass did not protect the best checkpoint	fixed
H-F10	The memory estimator was about 5 GB optimistic (7.96 to 8.12 against 13.09 GB)	fixed
H-F11	Consistency was grouped by scenario family (ceiling 0.600, 57 flips for a perfect model)	reported by group from H8 on
H-F12	The fabricated-citation heuristic flags gold answers (47.9% of test targets)	reported beside its floor
H-F13	Evidence coverage is vacuous (every response scores 1.0)	reported as vacuous
H-F14	Item-level intervals treat 349 dependent items as independent	cluster intervals from H8 on
L-F1	LoRA scoping for the vision-language adapter fails on Transformers 5 (7 of 14 modules landed in the vision tower)	open; the strict test is an expected failure
L-F2	The tokenizer class depends on `config.json` (442 against 492 tokens for the same text)	prompt tokens matched on 9 of 9 served prompts
L-F3	Evaluation resume lost killed sessions' work on the Drive mount	fixed; confirmed 2026-10-01 (172 generations resumed)
L-F4	A fresh start into a folder with checkpoints would delete the best one	open
L-F5	Checkpoint validation ignores optimizer and scheduler state	open
L-F6	`device_map: auto` split the model unevenly across two T4s (probe 10.16 GB on GPU 1 against an estimated 8.53; run peak 10.21 GB)	open; no effect on the run
L-F7	The stop reason misses a budget cut after a closed trace (6 of 349)	resolved in serving
L-F8	Evaluation reports no progress while running	open

F Artifact register

Three kinds of dataset identity appear in the records and are never interchangeable: the *release content hash*, computed by the data pipeline over a sealed release (`3cc9a74486c42e8e` for v0.0.6, `b53afa4216bf6973` for v0.0.7, as the changelog prints them); the harness *dataset hash*, computed over the loaded split files (`c7340f1d9cb31b1d1d7f0ade65d83740159a887f5d6cb4955820065076ffa875` for v0.0.6); and the deployment record's *dataset SHA-256* (`5d02a5afd7be18e57f13b7d86fa8c31a1b008dd9e1927f4fa506356649699c48` for v0.0.7). Likewise the test split (`a4decaaf029b227366846d44018b0b1b669d21f068800a9f3175e15ec4783980`) and the benchmark built from it (`a11ffad75f5147f9d0ddad7bad4bfc073dc730df2b19173ff642774233b4b266`) are different objects.

Table 18 gives each run's identity.

Table 18: Per-run identity. Commits are given as the frozen records cite them and as they appear in the public history after the rewrite described in Section 10; manifests hold the former. The pilot's configuration hash no longer reproduces at the current head because its base was pinned afterwards. Sources: the artifact audit, the run reports, the deployment record and `docs/experiments/README.md`.

Run	Identity
Ministral-8B pilot	config hash `3fbb3f90ed9662eef80136382 0ddaade0c24e591468447b030ef50d12b7806d3`; commit `2ad7c9a` (public `b64099b`), clean; dataset hash `c7340f1d...`; adapter `6d4a0a93d274160baeea65843f48d57df759cddc9d423f55a20d7c88659e473c`, 174,655,536 bytes
KLEOS Hermes v0.0.6	config hash `b2328857c6026dd7c904a6acadd00ec55a6ce5289bf6210e743615fce6d8d3e8`; commit `4cd76c4` (public `22ca978`), clean; dataset hash `c7340f1d...`; adapter `dc121fa36ce1409ca142e8da61a809f9386e32d0ef09c4214efa271d5b857b32`, 228,140,600 bytes; result files arm1 `71dee4c0...`, arm2 `428400f5...`, arm2 annotated `99fdcdc3...`
KLEOS Logos v0.0.1	config hash `18008c6716a58afc284c64eb7e1e96c9bfce34b8da2b8c489b6d59c0c45b6f71`; training commit `ec4f9e3` (public `81e3e9a`); arm1 evaluation commit `ed5a987` (public `e915060`); dataset hash `c7340f1d...`; adapter `f3e8dcdc09a3f3bb9f6fc4f64f867ff8b97834c955a03fcbde98a8068d5570a7`, 243,869,280 bytes; result files arm1 `a0b048b1...`, arm2 `09243630...`
KLEOS Logos v0.0.2	config hash `d1961583546b5761dd854cfe845838ca91a87ea6189d54be617de21085e8cfa9`; commit `a17ace7` (public `6d7e450`); release content hash `b53afa42...`; adapter `e49724f6554db662769fbe9b9431105dbc12f8f75cecd6e0e00728ea3f04dc4f`; result file arm2 `0dd74661...`, whose `benchmark_fingerprint` field is `c3d0d1f1...` (a hash stored by the harness, distinct from the benchmark SHA-256)

The Logos configuration hashes include the literal dataset and output paths used on the training hosts, which the record keeps for that reason. The pre-registration commits map as follows: H5 and H6 on 2026-09-15, `7145edd` to `a6bae0c`; H8 and the clustered-interval amendment on 2026-09-24, `ec4f9e3` to `81e3e9a`; D11 on 2026-09-29, `ed5a987` to `e915060`; H9 on 2026-10-06, `a17ace7` to `6d7e450`; the first commit, `fd87931` of 2026-08-14, is unchanged.

G Serving details and the smoke record

Table 19 records the host's rules and the checks behind Section 8.

Table 19: ZeroGPU facts as recorded on 2026-09-23 and the two-day Logos smoke record of 2026-10-07. Source: `docs/experiments/serving-verification-records.md`.

Item	Recorded value
GPU	half an NVIDIA RTX Pro 6000 Blackwell: 48 GB, compute capability 12.0
Daily quota	5 minutes of GPU time for a free account, 2 minutes for an unauthenticated caller; charged to the calling account; reset 24 hours after that account's first use
Admission and queue	admitted while remaining quota covers 1.5 times the requested duration; waits up to 60 s for a GPU; up to 2 ZeroGPU Spaces per account; no persistent storage
Duration rule	$\min(\max(\lceil \text{BASE}+\text{max_new_tokens}/\text{TOKENS_PER_SECOND} \rceil, \text{MIN}), \text{MAX})$ with defaults 10 s, 12 tokens/s, 15 s and 60 s; measured speeds of 12 (Hermes) and 11.0 (Logos) tokens/s left the defaults in place
Hermes, Colab check	adapter `dc121fa3...`; the package copy of the adapter configuration pins `04d8a905...`; tokenizer loaded without the regex fix, as trained; 9 of 9 byte-identical
Hermes, ZeroGPU (10 calls)	9 of 9 byte-identical, token counts equal, repeated call identical; 7 of 10 calls got a new worker; 73 to 131 s of the 300 s daily quota consumed; startup: download 2 s, load and quantization 41 s, packing 8.54 GB 3 s; peak CPU memory at startup not recorded
Logos, startup	download 1.9 s, load 57.1 s, host peak 30.6 GiB of 2,000 GiB, 9.20 GB packed for the GPU
Logos, day 1	7 of 9 prompts admitted before the quota refused the 8th; 6 of 7 identical; 161 to 174 GPU seconds; warm median 19.7 s, cold 29.9 s
Logos, day 2	2 of 2 identical (a tool-routing and a workspace-reasoning item); cold 25.2 s (236 tokens), warm 33.6 s (a 406-token answer); 56.5 to 58.8 GPU seconds
Logos, totals	8 of 9 identical in answer and trace; prompt tokens equal on 9 of 9; decisions agree on 8 of 9; 217.8 to 232.3 GPU seconds (24 to 26 per answer)
Logos, evaluation traces	349 of 349 answers traced; 0 traces left open; 0 empty answers or parse failures; mean 549 characters (median 540, maximum 840) against training traces of 327 to 878; completions mean 293.5 tokens, median 285, 95th percentile 413